# FSMELD: Accelerating Subsequence Matching under Elastic Measures via Amortized Sublinear Lower Bounds

Zemin Chao
Harbin Institute of Technology
Harbin, Heilongjiang, China
chaozm@hit.edu.cn

Zitong Li
Harbin Institute of Technology
Harbin, Heilongjiang, China
25S103343@stu.hit.edu.cn

John Paparrizos
Ohio State University
Columbus, Ohio, United States
john@paparrizos.org

Zhixin Qi
Harbin Institute of Technology
Harbin, Heilongjiang, China
qizhx@hit.edu.cn

Hongzhi Wang*
Harbin Institute of Technology
Harbin, Heilongjiang, China
wangzh@hit.edu.cn

## ABSTRACT

Elastic similarity measures provide robust similarity assessment for time series, but their dynamic-programming-based evaluation is costly for subsequence search. Existing acceleration techniques for subsequence matching predominantly target Dynamic Time Warping (DTW), leaving other elastic measures, such as Edit Distance with Real Penalty (ERP), Edit Distance on Real Sequences (EDR), Longest Common Subsequence (LCSS), and Move-Split-Merge (MSM), largely unexplored. In this paper, we introduce FSMELD, an FFT-accelerated lower-bounding framework for a broad class of similarity measures. Specifically, FSMELD formulates a generalized inner-product-based lower bound ($LB_{IP}$). By transforming the lower-bound computation into sliding inner-product operations, FSMELD leverages the Fast Fourier Transform (FFT) to reduce the amortized time complexity per candidate to $O(\log m)$ for subsequence matching, where $m$ is the query length. Extensive experiments on diverse datasets demonstrate that FSMELD achieves significant speedups over state-of-the-art exact approaches for time series subsequence matching across various elastic measures.

**PVLDB Reference Format:**
Zemin Chao, Zitong Li, John Paparrizos, Zhixin Qi, and Hongzhi Wang. FSMELD: Accelerating Subsequence Matching under Elastic Measures via Amortized Sublinear Lower Bounds. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/lzzt0123/FSMELD.

## 1 INTRODUCTION

Subsequence similarity search is a fundamental operation in time-series data management: given a query pattern, it identifies matching windows in a much longer sequence and supports downstream tasks such as retrieval, monitoring, motif discovery, and anomaly analysis [3, 11, 20]. A lock-step distance can change sharply when two otherwise similar patterns are locally shifted or stretched. Elastic measures address this problem by aligning observations in time, but the alignment also makes exact evaluation substantially more expensive.

Dynamic Time Warping (DTW) is the most widely used elastic similarity measure, but it is not a proxy for the entire family. In a comparison of nine similarity measures on 38 datasets, no single measure was uniformly superior across the evaluated datasets [10, 27]. Lines and Bagnall similarly found no significant overall accuracy difference among the individual elastic 1-NN classifiers in their benchmark, whereas their ensemble significantly outperformed the constituent classifiers [18]. The same complementarity appears in multivariate classification: among the measures evaluated by Shifaz et al. on 23 datasets, every measure but one achieved the highest accuracy on at least one dataset [21]. These are benchmark-specific findings rather than a universal dominance claim, but they show why a search engine restricted to DTW may exclude the best measure for a particular workload. Recent evaluations further highlight the practical value of MSM and other non-DTW elastic distances in clustering and multivariate analysis [9, 13].

This diversity is structural. Longest Common Subsequence (LCSS) emphasizes matched subsequences and tolerates unmatched observations [26]; Edit Distance on Real Sequences (EDR) uses thresholded edit costs to suppress small pointwise perturbations [8]; Edit Distance with Real Penalty (ERP) combines real-valued edit operations with local time shifting [7]; and Move-Split-Merge (MSM) assigns costs to move, split, and merge operations [22]. Supporting multiple measures is therefore not merely an API extension. FastEE, for example, addresses the computational cost of training ensembles of elastic measures for time-series classification [24].

The cost is especially acute in exact subsequence range search. For a query $Q \in \mathbb{R}^m$ and a target $S \in \mathbb{R}^n$, the search examines $n - m + 1$ overlapping candidates. A dynamic program restricted to a width-$w$ band typically costs $O(mw)$ per candidate, and local z-normalization must be applied consistently to every window when normalized matching is required. Exact systems therefore commonly use a filter–verify pipeline: a cheap admissible lower bound discards candidates whose bound exceeds the cutoff, and an exact evaluator processes the survivors. Research has improved both components. Strong linear-time pairwise bounds exist for DTW and have been generalized to other elastic measures, while early

*Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

abandoning and state pruning reduce the work of exact dynamic programming [12, 19, 28].

Although linear-time lower bounds have been generalized beyond DTW [5, 19], applying them independently to each overlapping length-$m$ candidate still incurs $O(m)$ cost per window. For example, the Generalized Lower Bounding (GLB) framework provides reusable per-series summaries for multiple elastic measures, yet its position-wise pairwise aggregation remains linear in $m$ [19]. For DTW, FSMDTW exploits neighboring windows for fixed-query subsequence search, and a separate motif bound attains amortized constant time [4, 6]. However, FSMDTW derives its inner-product formulation from a DTW-specific lower bound, and does not directly provide admissible instantiations for ERP, EDR, LCSS, or MSM due to their fundamentally different cost semantics. The remaining challenge is thus to provide a common admissible computation that amortizes lower-bound evaluation across overlapping, locally normalized windows for multiple elastic objectives.

To bridge this gap, we introduce FSMELD, a unified framework for accelerating exact subsequence range search under DTW, ERP, EDR, LCSS, and MSM. Rather than directly transforming the exact elastic distance into an inner product, FSMELD formulates a generalized inner-product-based lower bound, denoted by $LB_{IP}$. By reformulating lower-bound contributions as products of target-side envelope masks and query-side penalty functions, FSMELD transforms their aggregation into sliding inner-product operations. This transformation enables the application of the Fast Fourier Transform (FFT) to accelerate evaluation. By processing these inner products in batches over segments of length $\Theta(m)$ containing $\Theta(m)$ candidates, FSMELD achieves an amortized $O(\log m)$ lower-bound evaluation cost per candidate. Surviving candidates undergo further filtering and exact verification. Furthermore, the benefits of this inner-product-based lower bound extend beyond FFT acceleration. Because inner products support efficient incremental updates, the transformation allows distance bounds to be maintained in amortized $O(\log m)$ time for subsequence matching.

The main contributions of this paper are:

- We propose $LB_{IP}$, a generalized lower bound that transforms the evaluation of diverse elastic measures into sliding inner-product operations. By introducing a separable target-side mask and query-side penalty structure, $LB_{IP}$ safely decouples sequence morphology from amplitude, ensuring theoretical admissibility across structurally diverse cost semantics.
- We systematically analyze the mathematical properties of the $LB_{IP}$ formulation and propose strategies to optimize its dynamic thresholds $(t_U, t_L)$. Specifically, we design an exact method to compute the optimal parameters that maximize the pruning power for individual subsequence pairs, and develop a highly efficient heuristic algorithm that adapts to local baselines with bounded overhead.
- We propose FSMELD, a unified theoretical framework for accelerating subsequence matching under various elastic measures. By leveraging FFT on our inner-product formulation, we reduce the amortized time complexity of lower-bound computations to $O(\log m)$ per candidate, where $m$ is the query length. Furthermore, we provide metric-specific instantiations of this framework for four structurally diverse measures (ERP, EDR, LCSS, and MSM).
- Experiments on diverse datasets demonstrate that FSMELD maintains high pruning rates and achieves significant speedups over state-of-the-art exact dynamic-programming approaches and existing metric-specific bounding techniques.

## 2 BACKGROUND AND RELATED WORK

### 2.1 Definitions and Notation

We begin by formalizing the core concepts and notation used throughout this paper.

Definition 1. *(Time Series) A time series $S \in \mathbb{R}^n$ of length $n$ is an ordered sequence of $n$ real values, denoted $S = (S[1], S[2], \ldots, S[n])$, or equivalently $S = (s_1, s_2, \ldots, s_n)$ when subscript notation is used.*

Definition 2. *(Subsequence) A subsequence of a time series $S \in \mathbb{R}^n$ is a contiguous segment within $S$. We denote the subsequence starting at the $i$-th position as $S_{(i)} = (S[i], S[i+1], \ldots, S[i+m-1])$, where $m$ is the length of the subsequence.*

*2.1.1 Upper and Lower Envelopes.* Bounding envelopes are critical structures for defining distance lower bounds. For a given time series segment $T \in \mathbb{R}^\ell$, the upper envelope $\mathbb{U}^T \in \mathbb{R}^\ell$ and lower envelope $\mathbb{L}^T \in \mathbb{R}^\ell$ are defined with respect to a temporal constraint window $w \in \mathbb{N}^+$. Specifically, the $i$-th elements of the upper and lower envelopes of $T$ are

$$\mathbb{U}_i^T = \max_{\substack{1 \le j \le \ell \\ |i-j| \le w}} T[j], \qquad \mathbb{L}_i^T = \min_{\substack{1 \le j \le \ell \\ |i-j| \le w}} T[j]. \tag{1}$$

Throughout this paper, $ELD(X, Q)$ denotes the exact elastic distance evaluated under the same temporal constraint $w$ used by the envelopes and lower bounds. Accordingly, $w$ is a global parameter of $ELD$ and is omitted from its notation. For cost-minimizing dynamic programs, every state $(i, j)$ with $|i - j| > w$ is unreachable. For LCSS, a pair $(x_i, q_j)$ can contribute a match only if it satisfies both the amplitude tolerance and $|i - j| \le w$.

Definition 3. *(Subsequence Matching Problem under Elastic Similarity Measures) Given a target time series $S \in \mathbb{R}^n$, a query $Q \in \mathbb{R}^m$ with $m \le n$, an elastic measure $ELD(\cdot, \cdot)$, and a cutoff $\xi \ge 0$, subsequence matching returns the starting indices*

$$\mathcal{R}_\xi(S, Q) = \left\{ i \in \{1, \ldots, n-m+1\} : ELD\Big(S_{(i)}, Q\Big) \le \xi \right\}. \tag{2}$$

We assume a fixed cutoff for ease of presentation; FSMELD applies to every $\xi \ge 0$.

### 2.2 Elastic Similarity Measures

Elastic similarity measures address the temporal misalignment problem inherent in real-world time series data by allowing one-to-many or many-to-many point alignments. Table 1 summarizes the movement cost functions and final transformations for various widely adopted elastic measures.

DTW is the best-known elastic measure [1], utilizing dynamic programming to find the optimal alignment path that minimizes the cumulative distance. However, empirical evidence indicates that DTW is highly sensitive to noise and outliers. To address this vulnerability, EDR [8] and LCSS [26] introduce a matching tolerance

threshold $\epsilon$. EDR assigns a unit edit penalty when two aligned elements differ by more than $\epsilon$, whereas LCSS treats elements within $\epsilon$ as a match and maximizes the number of matched pairs. We convert the resulting LCSS similarity into a non-negative normalized distance, so that all measures considered in this paper share the same minimization objective. ERP [7] combines the $L_1$-norm distance with the string edit distance by introducing a constant reference gap value $g$, which allows it to satisfy the triangle inequality. [1] MSM [22] further introduces a fixed cost $c > 0$ for split and merge operations, offering distinct advantages in handling varied temporal phenomena.

Throughout this paper, *RAC*, *ELD*, all envelopes, and all lower bounds use the same temporal constraint $w$. For a selected elastic measure, let $RAC(i, j)$ (Raw Accumulated Cost) denote the minimum accumulated cost between prefixes $X[1:i]$ and $Q[1:j]$, with $RAC(0, 0) = 0$.

For DTW, EDR, ERP, and MSM, states outside the band are unreachable, i.e., $RAC(i, j) = +\infty$ for $|i - j| > w$. For every reachable interior state,

$$\begin{aligned} RAC(i,j) = \min \big\{ & RAC(i-1,j-1) + \mathbf{M}(x_i, q_j), \\ & RAC(i-1,j) + \mathbf{D}_V(x_i, q_j), \\ & RAC(i,j-1) + \mathbf{D}_H(x_i, q_j) \big\}. \end{aligned} \tag{3}$$

Boundary initialization is measure-specific. DTW and MSM use $RAC(i, 0) = RAC(0, j) = +\infty$ for $i, j > 0$. Inside the band, EDR uses $RAC(i, 0) = i$ and $RAC(0, j) = j$, whereas the squared-cost ERP uses $RAC(i, 0) = \sum_{k=1}^{i}(x_k - g)^2$ and $RAC(0, j) = \sum_{k=1}^{j}(q_k - g)^2$; their remaining boundary states are $+\infty$. The resulting distance is

$$ELD(X, Q) = \text{trans}(RAC(n, m)) \,. \tag{4}$$

LCSS is represented in the same RAC framework by minimizing the number of unmatched query elements. Its temporal constraint applies only to admissible matches. Let $\delta(i, j) = 0$ if $|x_i - q_j| \le \epsilon$ and $|i - j| \le w$, and $\delta(i, j) = 1$ otherwise. With $RAC(i, 0) = 0$ and $RAC(0, j) = j$,

$$\begin{aligned} RAC(i,j) = \min\{ & RAC(i-1,j),\ RAC(i,j-1) + 1, \\ & RAC(i-1,j-1) + \delta(i,j) \}. \end{aligned} \tag{5}$$

For the equal-length pairs considered in this paper,

$$d_{\text{LCSS}}(X, Q) = \frac{RAC(m, m)}{m} = 1 - \frac{\text{LCSS}(X, Q)}{m}. \tag{6}$$

Consequently, a smaller value always indicates greater similarity between the compared sequences, consistent with the minimization objective in Definition 3.

Despite their diverse mathematical formulations and proven effectiveness, these dynamic-programming-based measures suffer from quadratic time complexity [2, 3], which limits their direct applicability to massive time series datasets.

[1] The unconstrained ERP construction was originally formulated with $L_1$ costs, and its metric construction extends to $L_p$ norms. Our band-constrained $p = 2$ instantiation accumulates squared match and gap costs and returns the square root of the optimal accumulated cost.

[2] For equal-length pairs, LCSS is expressed as a non-negative loss-minimization RAC with $RAC(i, 0) = 0$ and $RAC(0, j) = j$. Hence, $RAC(m, m) = m - \text{LCSS}(X, Q)$ and $d_{\text{LCSS}}(X, Q) = RAC(m, m)/m$. The temporal constraint applies only to admissible LCSS matches.

## 2.3 Existing Lower Bounds and Acceleration Techniques

Exact elastic-distance search commonly follows a filter–verify pipeline: an admissible lower bound prunes candidates with no false dismissals, and exact dynamic programming verifies the survivors. DTW has the richest body of lower-bound research. Early methods such as $LB_{\text{Yi}}$ and $LB_{\text{Kim}}$ use global features, whereas $LB_{\text{Keogh}}$ compares one series to the warping envelope of the other [15, 16, 31]. Subsequent linear-time bounds, including $LB_{\text{Improved}}$, $LB_{\text{Enhanced}}$, and $LB_{\text{Webb}}$, improve the tightness–cost trade-off [17, 23, 28]. UCR Suite combines cascading bounds, reordered evaluation, and early abandoning into an exact DTW subsequence-search pipeline [20].

Beyond DTW, lower-bound development has been less uniform. Chen et al. developed ERP-specific bounds for the original ERP formulation [7]; Vlachos et al. developed lower-bounding techniques and no-false-dismissal indexing for LCSS [25, 26]; and FastEE introduced measure-specific bounds for MSM and TWED [24]. The original EDR work proposed three exact pruning filters, rather than a lower-bound distance conventionally denoted by $LB_{\text{EDR}}$ [8].

Paparrizos et al. [19] proposed the Generalized Lower Bounding (GLB) framework for multiple elastic measures, including the first reported general lower bound for EDR. After envelope preprocessing, however, GLB still scans each length-$m$ pair in $O(m)$ time. Closely related concurrent work proposed BGLB (Bipartite Graph-based Lower Bound), which strengthens GLB through a graph-derived augmentation term while retaining $O(m)$ time for equal-length pairs [5]. BGLB targets pairwise bound tightness, whereas FSMELD targets amortized evaluation over overlapping windows; BGLB+ELD is therefore our direct general-purpose filter–verify baseline.

DTW-specific work has also exploited reuse across related comparisons. An amortized $O(1)$ lower bound was developed for motif discovery [4], while FSMDTW evaluates a DTW bound at a batch-amortized $O(\log m)$ cost per sliding candidate [6]. FSMELD extends this computational direction beyond DTW by developing an admissible FFT-compatible formulation for ERP, EDR, LCSS, and MSM.

Learning-based surrogates offer a complementary speed–accuracy trade-off. NeuTS approximates target measures, including DTW and ERP, and supports approximate top-$k$ search [29]. Its learned score has not been established as an admissible lower bound and may therefore incur false dismissals. FSMELD retains exact verification.

# 3 THE UNIFIED INNER-PRODUCT LOWER BOUND ($LB_{IP}$) FRAMEWORK

This section presents the theoretical foundation of FSMELD: the unified inner-product-based lower-bound framework ($LB_{IP}$). The key idea behind $LB_{IP}$ is to decompose nonlinear elastic-distance evaluation into sliding inner-product operations. By transforming the lower-bound calculation into a form compatible with the Convolution Theorem, we bridge the gap between expensive dynamic programming and efficient frequency-domain acceleration.

We first present the mathematical formulation of $LB_{IP}$, followed by a rigorous proof of its lower-bounding property. We then provide an intuitive interpretation of the framework using a "water

**Table 1: Movement costs and final transformations for five elastic measures.**

| | DTW | LCSS[2] | EDR | ERP | MSM |
|---|---|---|---|---|---|
| $\mathbf{M}(x_i, q_j)$ | $(x_i - q_j)^2$ | $\begin{cases}0, & \lvert x_i - q_j\rvert \le \epsilon \text{ and } \lvert i-j\rvert \le w,\\ 1, & \text{otherwise}\end{cases}$ | $\begin{cases}0, & \lvert x_i - q_j\rvert \le \epsilon,\\ 1, & \text{otherwise}\end{cases}$ | $(x_i - q_j)^2$ | $\lvert x_i - q_j\rvert$ |
| $\mathbf{D_V}(x_i, q_j)$ | $(x_i - q_j)^2$ | 0 | 1 | $(x_i - g)^2$ | $\begin{cases}c, & x_{i-1} \le x_i \le q_j \text{ or } x_{i-1} \ge x_i \ge q_j,\\ c + \min\{\lvert x_i - x_{i-1}\rvert,\ \lvert x_i - q_j\rvert\}, & \text{otherwise}\end{cases}$ |
| $\mathbf{D_H}(x_i, q_j)$ | $(x_i - q_j)^2$ | 1 | 1 | $(q_j - g)^2$ | $\begin{cases}c, & q_{j-1} \le q_j \le x_i \text{ or } q_{j-1} \ge q_j \ge x_i,\\ c + \min\{\lvert q_j - q_{j-1}\rvert,\ \lvert q_j - x_i\rvert\}, & \text{otherwise}\end{cases}$ |
| $\text{trans}(\cdot)$ | $\sqrt{RAC(n,m)}$ | $RAC(m,m)/m$ | $RAC(n,m)$ | $\sqrt{RAC(n,m)}$ | $RAC(n,m)$ |

level" metaphor, show that the resulting formulation admits amortized sublinear-time evaluation, and demonstrate how it can be instantiated across diverse elastic measures.

## 3.1 Formulation of $LB_{IP}$ for Elastic Measures

For conciseness, we denote the indicator function for a predicate $P$ as:

$$\mathbf{1}_{\{P\}} = \begin{cases}1 & \text{if } P \text{ is true}\\ 0 & \text{otherwise}\end{cases} \tag{7}$$

The proposed $LB_{IP}$ framework constructs a lower bound by independently evaluating the dissimilarity contributions from the upper and lower envelopes of the target sequence.

Definition 4. *Given time series $X, Q \in \mathbb{R}^m$ and a cost-based elastic measure $ELD(X, Q)$, suppose its envelope lower-bound contribution can be expressed through a matching cost $M$ and a deletion (or gap) penalty $D$ (defined below). We define the inner-product-based lower bound $LB_{IP}$ for any thresholds $t_U, t_L \in \mathbb{R}$ as:*

$$\begin{aligned} LB_{IP}(X, Q, t_U, t_L) = \text{trans}\Bigg( &\sum_{i=1}^{m} g_U(i, t_U) \cdot h_U(i, t_U) \\ &+ \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L)\Bigg) \end{aligned} \tag{8}$$

*where the valley mask functions $g_U$ and $g_L$ are defined as:*

$$g_U(i,t) = \mathbf{1}_{\{t \ge \mathbb{U}_i^X\}}, \quad g_L(i,t) = \mathbf{1}_{\{t \le \mathbb{L}_i^X\}} \tag{9}$$

*For cost-based formulations, the protrusion penalty functions $h_U$ and $h_L$ are defined as:*

$$h_U(i,t) = \begin{cases}\min\{M(q_i,t), D(q_i)\}, & t < q_i\\ 0 & otherwise\end{cases} \tag{10}$$

$$h_L(i,t) = \begin{cases}\min\{M(q_i,t), D(q_i)\}, & q_i < t\\ 0 & otherwise\end{cases} \tag{11}$$

In this formulation, $M(\cdot)$ represents the distance-dependent matching cost function, and $D(\cdot)$ denotes the corresponding deletion or gap penalty for the cost-based measures. LCSS does not use this $M$–$D$ interpretation; instead, its protrusion penalty function is instantiated separately as a binary function in Section 3.5.

## 3.2 The Lower Bound Property of $LB_{IP}$

The validity of the FSMELD framework requires that $LB_{IP}$ consistently underestimate the true elastic distance, regardless of the threshold parameters. We formalize this property in the following theorem.

Theorem 1. *Given time series $X \in \mathbb{R}^m$ and $Q \in \mathbb{R}^m$, for any cost-based elastic measure $ELD(X, Q)$ satisfying Definition 4, and any given thresholds $t_U, t_L \in \mathbb{R}$:*

$$ELD(X, Q) \ge LB_{IP}(X, Q, t_U, t_L) \tag{12}$$

Proof. A lower bound for $ELD(X, Q)$ can be constructed from two symmetric components: $\Gamma_U$, representing the dissimilarity contribution between the upper envelope $\mathbb{U}^X$ and $Q$, and $\Gamma_L$, representing the contribution between the lower envelope $\mathbb{L}^X$ and $Q$. Therefore,

$$RAC(X, Q) \ge \Gamma_U + \Gamma_L \tag{13}$$

where the final distance is obtained by applying a monotonically non-decreasing transformation function $\text{trans}(\cdot)$ to the non-negative raw accumulated cost. Following Paparrizos et al. [19], $\Gamma_U$ can be lower-bounded as:

$$\Gamma_U \ge \sum_{i=1}^{m} \begin{cases}\min\{M(q_i, \mathbb{U}_i^X), D(q_i)\}, & \mathbb{U}_i^X < q_i\\ 0, & \text{otherwise}\end{cases} \tag{14}$$

Thus, we derive a weaker but valid lower bound by introducing a threshold $t_U \in \mathbb{R}$ and restricting attention to regions where the envelope lies at or below this threshold (i.e., $\mathbb{U}_i^X \le t_U$). For the standard elastic measures considered, the matching cost $M(q_i, \cdot)$ is inherently non-negative distance functions. Consequently, $M(q_i, t)$ is non-negative and monotonically non-increasing with respect to $t$ exclusively within the interval $\mathbb{U}_i^X \le t < q_i$. Replacing $\mathbb{U}_i^X$ with $t_U$ therefore logically guarantees $M(q_i, \mathbb{U}_i^X) \ge M(q_i, t_U)$. Encoding this condition as the indicator mask $g_U(i, t_U) = \mathbf{1}_{\{t_U \ge \mathbb{U}_i^X\}}$ and substituting the penalty $h_U(i, t_U)$ yields the following inequality:

$$\begin{aligned} \Gamma_U &\ge \sum_{i=1}^{m} \begin{cases}\min\{M(q_i, t_U), D(q_i)\}, & \mathbb{U}_i^X \le t_U < q_i\\ 0, & \text{otherwise}\end{cases} \\ &= \sum_{i=1}^{m} \mathbf{1}_{\{t_U \ge \mathbb{U}_i^X\}} \times \begin{cases}\min\{M(q_i, t_U), D(q_i)\}, & t_U < q_i\\ 0, & \text{otherwise}\end{cases} \\ &= \sum_{i=1}^{m} g_U(i, t_U) \times h_U(i, t_U) \end{aligned} \tag{15}$$

By symmetry, applying identical logic to the lower envelope yields $\Gamma_L \geq \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L)$. Summing these two independent components directly yields:

$$RAC(X, Q) \geq \sum_{i=1}^{m} g_U(i, t_U) \cdot h_U(i, t_U) + \sum_{i=1}^{m} g_L(i, t_L) \cdot h_L(i, t_L) \quad (16)$$

Because the transformation function $\text{trans}(\cdot)$ associated with the elastic measure *ELD* is monotonically non-decreasing, applying it to both sides preserves the inequality. Thus, we establish that:

$$\begin{aligned} ELD(X, Q) &\geq \text{trans}\left(\sum_{i=1}^{m} g_U \cdot h_U + \sum_{i=1}^{m} g_L \cdot h_L\right) \\ &= LB_{IP}(X, Q, t_U, t_L) \end{aligned} \quad (17)$$

which completes the proof. □

## 3.3 Intuitive Explanation of the $LB_{IP}$ Mechanism

To provide an intuitive understanding of the mathematical formulation, we use a "water level and penalty filtering" metaphor as illustrated in Figure 1. The threshold $t_U$ acts as a global "water level" that decouples the target series' morphology from the query series' amplitude.

Specifically, $g_U(i, t_U)$ serves as a binary mask identifying the "valleys" within the target sequence. If the upper envelope $\mathbb{U}_i^X$ falls at or below the water level $t_U$, the position marks a region in which a penalty may be incurred (yielding $g_U = 1$). Otherwise, the mask is 0.

On the query side, $h_U(i, t_U)$ quantifies the inherent "protrusion penalty" of $Q$. If a query element $q_i$ acts as a protrusion rising above $t_U$, forcing its alignment with any submerged target element incurs an unavoidable baseline cost. The function $h_U$ precisely captures this minimum guaranteed penalty, which represents either the cost of matching down to the water level $M(q_i, t_U)$ or the fixed deletion cost $D(q_i)$.

Consequently, the framework transforms the lower-bound computation for complex elastic alignments into a decoupled sliding inner product. A valid penalty is accumulated only when a protruding query element ($h_U > 0$) aligns with a guaranteed target valley ($g_U = 1$). This decoupling mechanism enables the application of the Convolution Theorem, thereby yielding amortized sublinear time complexity.

## 3.4 Amortized Sublinear Time Evaluation of $LB_{IP}$

The primary motivation for decoupling each elastic measure into a valley mask $g$ and a protrusion penalty $h$ is to bridge the gap between complex morphological alignments and linear convolutions. Crucially, given a threshold $t$, the masks $g_U, g_L$ depend exclusively on the target sequence $X$, while the penalties $h_U, h_L$ depend exclusively on the query $Q$. The $LB_{IP}$ formulation exploits this strict independence to transform elastic distance lower-bound computations into a pure inner-product operation, thereby naturally enabling amortized sublinear-time subsequence matching.

In subsequence matching, this bound must be evaluated for every length-$m$ subsequence within a target segment $T \in \mathbb{R}^\ell$ of length

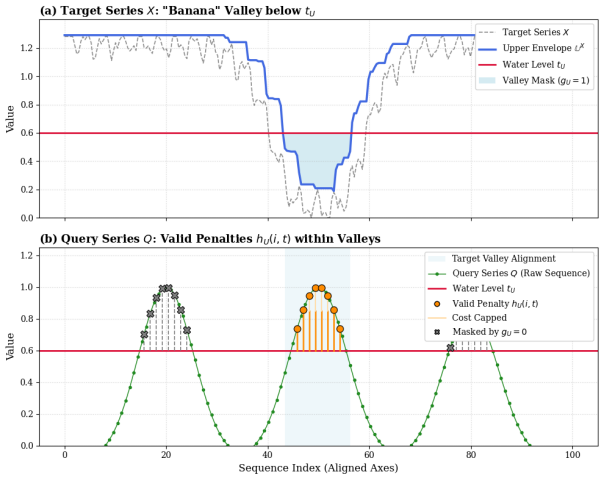



**Figure 1: Water-level interpretation of the upper-envelope penalty contribution in $LB_{IP}$ (denoted by $\Delta_U(t_U)$ in Section 4). (a) $t_U$ activates target valleys through $g_U$. (b) Only query protrusions aligned with active valleys contribute to the bound.**

$\ell > m$. Let $k \in \{1, 2, \ldots, \ell - m + 1\}$ denote the sliding window starting index. For arbitrary user-defined thresholds $t_U$ and $t_L$, we first construct global binary mask vectors $A, A' \in \{0, 1\}^\ell$ for the entire target segment and metric-specific protrusion penalty vectors $B, B' \in \mathbb{R}^m$ for the query sequence $Q$. This construction is formulated as:

$$A = (g_U(1, t_U), g_U(2, t_U), \ldots, g_U(\ell, t_U)) \quad (18)$$

$$B = (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U)) \quad (19)$$

$$A' = (g_L(1, t_L), g_L(2, t_L), \ldots, g_L(\ell, t_L)) \quad (20)$$

$$B' = (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L)) \quad (21)$$

Let $A_{(k)}$ and $A'_{(k)}$ denote the contiguous length-$m$ subvectors of $A$ and $A'$ starting at index $k$, which represent the active masks for the $k$-th sliding window. Let $\langle \cdot, \cdot \rangle$ denote the standard vector inner product. The expression of $LB_{IP}$ for the $k$-th candidate subsequence $T_{(k)}$ can be transformed from its piecewise sum into two vector inner products:

$$LB_{IP}(T_{(k)}, Q, t_U, t_L) = \text{trans}\left(\langle A_{(k)}, B \rangle + \langle A'_{(k)}, B' \rangle\right) \quad (22)$$

By recasting the alignment into vectors, evaluating the lower bounds across all $N_w = \ell - m + 1$ subsequences becomes mathematically equivalent to a discrete linear convolution between the target mask array and the reversed query penalty array. Rather than using an expensive sliding-window approach, we encapsulate this via the Convolution Theorem and FFT into a Fast Sliding Inner Product function (SIP, Algorithm 1), which simultaneously evaluates all inner products in $O(\ell \log \ell)$ time. Following established practices [6, 30], setting the segment length to $\ell = \Theta(m)$ reduces the amortized time cost per candidate to $O(\log m)$.

**Algorithm 1:** SIP$(T, X)$

**Input:** $T \in \mathbb{R}^{\ell}, X \in \mathbb{R}^{m}, (\ell > m)$.
**Output:** The inner product between $X$ and $T_{(i)}$ for $i \in \{1, 2, \ldots, \ell - m + 1\}$.

$X_{pad} \leftarrow$ reverse $X$ and pad with $\ell - m$ trailing zeros;
$COV \leftarrow IDFT\left(DFT(T) \odot DFT(X_{pad})\right)$ ; // $O(\ell \log \ell)$
**return** the real part of $COV[m-1 : \ell]$.

## 3.5 Instantiating FSMELD for Specific Elastic Distances

To demonstrate the generality of the $LB_{IP}$ framework, we instantiate it for four widely used yet computationally expensive elastic measures: ERP, EDR, LCSS, and MSM.

Applying the FSMELD framework to a specific elastic measure requires mathematically formulating its corresponding valley mask function $g_U(i,t)$ and protrusion penalty function $h_U(i,t)$. Table 2 summarizes the mappings required to integrate these measures into our unified architecture. All these measures can be unified under the $LB_{IP}$ paradigm. This unification, however, requires careful metric-specific adaptations. For EDR and LCSS, the binary protrusion penalties incorporate the matching tolerance $\epsilon$. For MSM, the protrusion penalty is capped at $c$, the base split/merge cost.

*LCSS-specific instantiation.* For LCSS, the binary penalties in Table 2 certify query positions that cannot participate in any admissible match; they do not represent edit costs. The upper and lower contributions cannot certify the same query position, so their sum counts distinct unmatched query positions. For equal-length pairs, $\text{trans}(z) = z/m$, yielding

$$LB_{IP}(X, Q, t_U, t_L) \le d_{\text{LCSS}}(X, Q). \tag{23}$$

The proof is given in Appendix A of our extended technical report[3].

**Table 2: Metric-specific upper-side penalties for FSMELD.**

| Metric | Penalty $h_U(i,t)$ |
|---|---|
| **ERP** | $\min\{(q_i - t)^2, (q_i - g)^2\}$ |
| **EDR** ($\epsilon$) | $\mathbf{1}_{\{q_i > t+\epsilon\}}$ |
| **LCSS** ($\epsilon$) | $\mathbf{1}_{\{q_i > t+\epsilon\}}$ |
| **MSM** ($c$) | $\min\{c, q_i - t\}$ |

As indicated in the final column of Table 2, transforming the raw time series into the required mask and penalty functions incurs at most amortized $O(1)$ time per element. This ensures that the metric-specific preprocessing step does not become a bottleneck in the sublinear matching algorithm.

# 4 THRESHOLD OPTIMIZATION FOR THE UNIFIED LOWER BOUND

The proposed $LB_{IP}$ framework enables lower-bound evaluation in amortized $O(\log m)$ time per candidate for any given thresholds $t_U$ and $t_L$. However, its effectiveness still depends on value of $t_U$ and $t_L$. This section studies how to select these thresholds for all candidates within a target segment. We first formulate the optimization objective and identify a finite candidate set, then describe exact threshold computation and an efficient heuristic approximation that preserves the amortized sublinear cost of FSMELD.

## 4.1 Objective Formulation

Given a target segment $T \in \mathbb{R}^{\ell}$ and a query sequence $Q$, we aim to select shared thresholds $(t_U, t_L)$ that optimize pruning effectiveness of the proposed $LB_{IP}$ framework. To this end, a natural and effective optimization objective is to directly maximize the cumulative penalty contributions across all overlapping windows.

Definition 5 (Threshold Optimization Objective). *Let $k \in [1, \ell - m + 1]$ denote the sliding window offset. The optimal thresholds $t_U^*$ and $t_L^*$ independently maximize the cumulative penalty components across all windows:*

$$\begin{aligned} t_U^* &\in \arg\max_{t \in \mathbb{R}} \sum_{k=1}^{\ell-m+1} \Delta_U(T_{(k)}, Q, t), \\ t_L^* &\in \arg\max_{t \in \mathbb{R}} \sum_{k=1}^{\ell-m+1} \Delta_L(T_{(k)}, Q, t). \end{aligned} \tag{24}$$

*where $\Delta_U$ and $\Delta_L$ represent the penalty contributions from the upper and lower envelopes for the subsequence $T_{(k)}$, computed using the shared segment envelopes $\mathbb{U}^T$ and $\mathbb{L}^T$, respectively.*

Recall that $LB_{IP}$ consists of two symmetric components: the dissimilarity contribution $\Delta_U(T_{(k)}, Q, t_U)$ derived from the upper envelope, and $\Delta_L(T_{(k)}, Q, t_L)$ derived from the lower envelope. Since the evaluation of the upper and lower envelopes relies on independent threshold variables, the joint optimization of maximizing the total distance penalty naturally decouples into two independent and symmetric subproblems. For conciseness, the remainder of this section focuses primarily on optimizing the upper threshold $t_U$. [4]

## 4.2 Properties of the Objective Function

By analyzing the penalty function defined in Section 4.1, we demonstrate that the continuous search space for the optimal threshold can be safely reduced to a discrete set of candidate values, as formalized in the following theorem.

Theorem 2 (Discrete Search Space for Optimal Threshold). *For a target segment $T$ and query sequence $Q$, an optimal threshold $t_U^*$ for the cumulative objective in Definition 5 can be selected such that*

$$t_U^* \in \{\mathbb{U}_i^T\}_{i=1}^{\ell}. \tag{25}$$

Proof Sketch. By definition, the upper penalty for a fixed window $T_{(k)}$ is evaluated as $\Delta_U(T_{(k)}, Q, t_U) = \sum_{i=1}^{m} g_U(i+k-1, t_U) h_U(i, t_U)$. The valley mask $g_U(i+k-1, t_U) = \mathbf{1}_{\{t_U \ge \mathbb{U}_{i+k-1}^T\}}$ acts as an indicator function, introducing stepwise discontinuities at the corresponding target envelope values. Figure 2 illustrates an individual contribution, denoted by $\delta_U^{(i)}(t) = g_U(i+k-1, t) h_U(i, t)$, with $q = q_i$ and the envelope marker corresponding to $\mathbb{U}_{i+k-1}^T$.

[3] The extended version containing all appendices and additional proofs is available at https://github.com/lzzt0123/FSMELD

[4] For an individual subsequence pair, the thresholds maximizing $LB_{IP}$ can be computed in $O(m \log m)$ time (See Appendix B of the extended report).

Within any interval between two consecutive envelope values, the set of activated target valleys remains fixed. Concurrently, for any active query element, the protrusion penalty $h_U(i, t_U)$ is monotonically non-increasing as the threshold $t_U$ rises.

Because $\Delta_U(T_{(k)}, Q, t_U)$ is non-increasing within each such interval for every window, the same property holds for the cumulative sum over all windows. Moving a threshold to the left envelope endpoint cannot decrease this cumulative sum. Therefore, an optimal threshold $t_U^*$ maximizing the objective can always be selected from the discrete envelope set. □

## 4.3 Exact Computation of Optimal Thresholds

Given a target segment $T \in \mathbb{R}^\ell$ and a query sequence $Q \in \mathbb{R}^m$, this subsection presents the method that computes the optimal threshold $t_U^*$ in $O(m^2 + \ell \log \ell)$ time.

*4.3.1 Objective Decomposition.* By Theorem 2, the optimal upper threshold $t_U^* \in \{\mathbb{U}_i^T\}_{i=1}^{\ell}$. Assuming $\ell \geq 2m$, as in the segment configuration used by FSMELD, we have

$$\begin{aligned}\sum_{k=1}^{\ell-m+1} \Delta_U(T_{(k)}, Q, t_U) &= \sum_{k=1}^{\ell-m+1} \sum_{i=1}^{m} g_U(i+k-1, t_U) \times h_U(i, t_U) \\ &= p_1(t_U) + p_2(t_U) + p_3(t_U)\end{aligned} \tag{26}$$

The core term $p_1(t_U)$ collects target positions that occur with every query position across the sliding windows:

$$p_1(t_U) = \left[\sum_{j=m}^{\ell-m+1} g_U(j, t_U)\right] \left[\sum_{i=1}^{m} h_U(i, t_U)\right]. \tag{27}$$

The boundary terms $p_2(t_U)$ and $p_3(t_U)$ capture the partial overlaps at the left and right edges of the segment:

$$p_2(t_U) = \sum_{j=1}^{m-1} g_U(j, t_U) \left[\sum_{i=\max\{1, j-\ell+m\}}^{j} h_U(i, t_U)\right], \tag{28}$$

$$p_3(t_U) = \sum_{j=\ell-m+2}^{\ell} g_U(j, t_U) \left[\sum_{i=j-(\ell-m)}^{\min\{m, j\}} h_U(i, t_U)\right]. \tag{29}$$

*4.3.2 Exact Computation.* This subsection presents the method that computes the optimal threshold $t_U^*$ in $O(m^2 + \ell \log \ell)$ time. The detailed pseudocode for this exact procedure is deferred to Appendix C.

Theorem 3. *For a target segment $T$ of length $\ell \geq 2m$ and a query $Q$ of length $m$, the exact optimal thresholds $t_U^*$ for the objective in Definition 5 can be computed in $O(m^2 + \ell \log \ell)$ time.*

Proof Sketch. Each query penalty is piecewise polynomial (degree $\leq 2$) with $O(1)$ breakpoints. Sorting the $O(\ell + m)$ envelope and query-penalty breakpoints requires $O(\ell \log \ell)$ time.

During the sweep, the active core masks and aggregate polynomial coefficients of $\sum_i h_U(i, t)$ are incrementally maintained. Each transition updates these core statistics in $O(1)$ time, enabling $O(1)$ evaluation of $p_1(t)$.

Expanding $p_2(t)$ and $p_3(t)$ yields $O(m^2)$ products $g_U(j, t)h_U(i, t)$, whose polynomial coefficients are also aggregated. Each product is processed once upon mask activation and updated at most $O(1)$ times at the breakpoints of its associated query penalty. All updates use the already-sorted events, so maintaining the boundary terms requires $O(m^2)$ total work.

At a shared event value, query transitions are processed before mask activations to respect the penalty values at the breakpoints. After these updates, the full objective is evaluated in $O(1)$ time using the maintained aggregates. By Theorem 2, checking all envelope thresholds guarantees an exact maximizer, yielding an overall $O(m^2 + \ell \log \ell)$ complexity. The lower-threshold computation follows symmetrically. □

Although setting $\ell = Km^2$ ($K \geq 2$) reduces the amortized threshold optimization cost to $O(\log m)$ per candidate, it is difficult to select a single suitable threshold across such long segments. Therefore, to make shorter segments ($\ell = \Theta(m)$) computationally feasible in batch processing, Section 4.4 introduces a heuristic strategy that applies the same incremental penalty-maintenance principle to the core term $p_1(t)$.

## 4.4 Fast $O(\ell \log \ell)$ Heuristic Optimization Strategy

*4.4.1 Heuristic Optimization Objective.* To reduce threshold-selection overhead for segments of length $\ell = \Theta(m)$, we omit the boundary terms $p_2(t)$ and $p_3(t)$ and maximize $p_1(t)$ alone. The selected threshold maximizes the core objective and serves as a heuristic choice for the full cumulative penalty. The boundary terms are omitted only when selecting thresholds; subsequent $LB_{IP}$ evaluation still uses all candidate positions. Since $LB_{IP}$ is admissible for any thresholds, this choice preserves exact subsequence matching. The resulting cost–pruning trade-off is evaluated in Section **??**.

Applying the monotonicity argument in Section 4.2 to $p_1(t)$, a maximizer can be selected from the core envelope values. Let $\mathcal{U} \in \mathbb{R}^\ell$ be the upper-bounding envelope of a segment. Algorithm 2 extracts $\mathcal{U}_{core} = \mathcal{U}[m : \ell - m + 1]$ and sorts its distinct values to construct the candidate set $\mathcal{T}$. The frequency of each value is precomputed, allowing the activation count $G_U$, the first factor of $p_1(t)$, to be updated without repeatedly scanning the core envelope.

*4.4.2 Dynamic State Tracking and Cost Maintenance.* The second factor of $p_1(t)$ is the total query protrusion penalty $\Delta_Q(Q, t)$. The heaps track only query penalties; the target-side masks are counted separately by $G_U$. Let $\tau(q_i)$ denote the end of the fixed-penalty branch. As $t$ increases, each query element $q_i$ belongs to one of three states:

- **State 1 (Fixed Penalty):** When $t < \tau(q_i)$, the contribution is $D(q_i)$. We maintain its index in a min-heap $H_2$, ordered by $\tau(q_i)$.
- **State 2 (Post-transition Penalty):** When $\tau(q_i) \leq t < q_i$, the element follows the post-transition branch of $h_U(i, t)$: quadratic for ERP and DTW, linear for MSM, and zero for EDR and LCSS. We maintain its index in a min-heap $H_1$, ordered by $q_i$.
- **State 3 (Submerged):** When $t \geq q_i$, the element contributes zero and is absent from both heaps.

DTW has no fixed-penalty state; we use $\tau(q_i) = -\infty$ in the algorithm. For EDR and LCSS, the fixed branch of $h_U(i, t)$ has value 1.

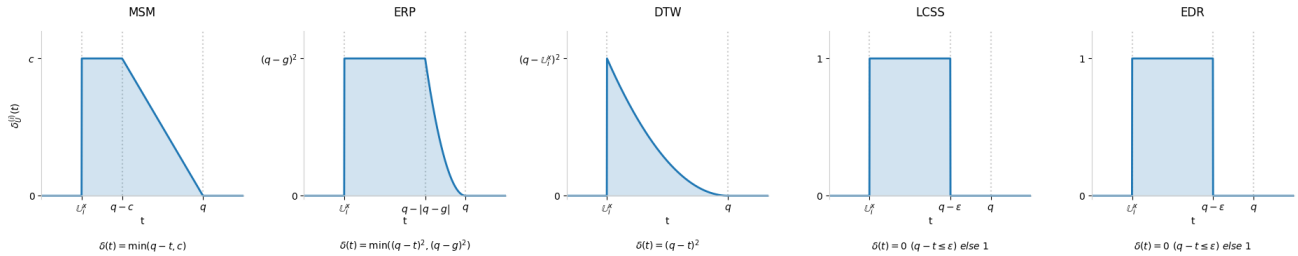



**Figure 2: The relationship between $t$ and the dissimilarity score contribution.**

For LCSS, $D(q_i) = 1$ is used solely as a bookkeeping convention in this threshold-selection procedure: it represents a certified lost match, not an LCSS deletion cost. The zero-contribution entries of EDR and LCSS are retained in $H_1$ to use the same state-transition procedure for all measures. The transition boundaries are summarized in Table 3.

**Table 3: State transition boundaries $\tau(q_i)$ for core elastic measures (assuming $q_i > t$).**

| Elastic Metric | State-transition condition | $\tau(q_i)$ (Condition of $H_2 \to H_1$) |
|---|---|---|
| DTW | no $H_2$ | no condition |
| LCSS | $q_i - t = \epsilon$ | $q_i - \epsilon$ |
| EDR | $q_i - t = \epsilon$ | $q_i - \epsilon$ |
| ERP | $q_i - t = \lvert g - q_i \rvert$ | $q_i - \lvert q_i - g \rvert$ |
| MSM | $q_i - t = c$ | $q_i - c$ |

At any given threshold $t$, the total query protrusion penalty is

$$\begin{aligned}\Delta_Q(Q,t) &= \sum_{i=1}^{m} h_U(i,t) \\ &= \sum_{i\in H_1} h_U(i,t) + \sum_{i\in H_2} D(q_i).\end{aligned} \tag{30}$$

Here, summations over $H_1$ and $H_2$ refer to their stored query indices. We maintain the moments $\mathcal{M}_\alpha = \sum_{i\in H_1} q_i^\alpha$, where $\mathcal{M}_0 = |H_1|$. The contribution of $H_1$ can then be evaluated in constant time via the polynomial function $f_p$. For MSM,

$$\sum_{i\in H_1} h_U(i,t) = \mathcal{M}_1 - t|H_1|. \tag{31}$$

For ERP and DTW,

$$\sum_{i\in H_1} h_U(i,t) = \mathcal{M}_2 - 2t\mathcal{M}_1 + t^2|H_1|. \tag{32}$$

The right-hand sides define $f_1(\mathcal{M}_1, |H_1|, t)$ and $f_2(\mathcal{M}_2, \mathcal{M}_1, |H_1|, t)$, respectively. Here, $p$ denotes the highest moment order required by the measure: $p = 2$ for ERP and DTW, and $p = 1$ for MSM. EDR and LCSS require only counts ($p = 0$), with zero contribution from $H_1$, i.e., $f_0(|H_1|, t) = 0$. For $p = 0$, the positive-order moment arguments are omitted. The fixed-penalty sum $\sigma_D = \sum_{i\in H_2} D(q_i)$ is maintained separately.

*4.4.3 Threshold Selection Algorithm and Complexity.* Algorithm 2 initializes the query states at $t_0 = \mathcal{T}_1$. At each candidate threshold, it first transfers elements with $\tau(q_i) \le t$ from $H_2$ to $H_1$, then removes elements with $q_i \le t$ from $H_1$. After these updates, $\Delta_Q(Q,t) = \sum_i h_U(i,t)$ and $G_U$ equals the number of core envelope values at most $t$. Their product therefore gives $p_1(t)$ exactly. Equality must be processed before evaluation, particularly for EDR and LCSS, whose penalties drop to zero at $t = q_i - \epsilon$. The best candidate in $\mathcal{T}$ therefore maximizes the heuristic objective.

Sorting the core envelope values and recording their frequencies requires $O(\ell \log \ell)$ time. Each query element enters and exits each heap at most once, giving $O(m \log m)$ heap maintenance. The count operation retrieves the corresponding precomputed frequency in $O(1)$ time. After state updates, evaluating each candidate threshold takes $O(1)$ time. Since $m \le \ell$, the total threshold-selection cost is $O(\ell \log \ell)$. With $\ell = \Theta(m)$, this gives an amortized $O(\log m)$ cost per candidate subsequence, preserving the amortized sublinear lower-bound evaluation cost of FSMELD. The lower threshold is obtained symmetrically.

## 5 THE PROPOSED SUBSEQUENCE MATCHING ALGORITHM

This section integrates the $LB_{IP}$ framework into a complete subsequence matching algorithm, termed FSMELD, which employs a cascaded pruning architecture. By leveraging the FFT, FSMELD achieves an amortized sublinear complexity of $O(\log m)$ per candidate for lower-bound evaluation, filtering out the vast majority of unpromising candidates before they reach the expensive exact-verification stage.

Time series subsequence matching typically requires subsequence normalization [14]. Specifically, each length-$m$ subsequence of $S$ must be z-normalized to have zero mean and unit variance. While Appendix D details the FSMELD algorithm applied directly to raw time series data, the remainder of this section focuses on the z-normalized matching extension.

### 5.1 Conservative Envelopes for z-Normalized Subsequences

Consider a target segment $T \in \mathbb{R}^\ell$ and its length-$m$ candidate windows. Z-score normalization is frequently required to achieve scale and baseline invariance in time series mining applications. For a subsequence $X$, its normalized form $\hat{X}$ is computed as $\hat{X}[i] = \frac{X[i]-\mu}{\sigma}$, where $\mu$ and $\sigma$ are the mean and standard deviation of $X$,

**Algorithm 2:** OptUpperThreshold($\mathcal{U}, Q$)

**Input:** $\mathcal{U} \in \mathbb{R}^{\ell}$, $Q \in \mathbb{R}^{m}$, $(\ell \geq 2m)$.
**Output:** The threshold value for $LB_{IP}$.

$\mathcal{U}_{core} \leftarrow \mathcal{U}[m : \ell - m + 1]$ ;
$\mathcal{T} \leftarrow$ Extract unique elements from $\mathcal{U}_{core}$ and record their counts ;
Sort $\mathcal{T}$ in ascending order ;
$H_1, H_2 \leftarrow$ Initialize two empty min-heaps ;
$G_U \leftarrow 0$; $t_0 \leftarrow \mathcal{T}_1$; $\sigma_D \leftarrow 0$; $\mathcal{M}_0, \mathcal{M}_1, \ldots, \mathcal{M}_p \leftarrow 0$ ;
**for** $j = 1$ *to* $m$ **do**
  **if** $q_j \leq t_0$ **then**
    continue ; // $j \notin H_1 \cup H_2$
  **if** $\tau(q_j) > t_0$ **then**
    $H_2.insert(\tau(q_j), j)$ ; // Tuple of (thresh, idx)
    $\sigma_D \leftarrow \sigma_D + D(q_j)$ ; // Contribution of $H_2$.
  **else**
    $H_1.insert(q_j, j)$ ; // Tuple of (thresh, idx)
    **for** $\alpha = 0$ *to* $p$ **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_j)^\alpha$ ; // Contribution of $H_1$.
**for** $i = 1$ *to* $|\mathcal{T}|$ **do**
  $t \leftarrow \mathcal{T}_i$ ;
  $G_U \leftarrow G_U + \text{count}(\mathcal{U}_{core}, \mathcal{T}_i)$ ;
  **while** ***not*** $H_2.empty()$ ***and*** $H_2.top().thresh \leq t$ **do**
    $k \leftarrow H_2.top().\text{idx}$ ;
    $\sigma_D \leftarrow \sigma_D - D(q_k)$ ; // $H_2 \leftarrow H_2 \setminus \{k\}$
    $H_1.insert(q_k, k)$ ; // $H_1 \leftarrow H_1 \cup \{k\}$
    **for** $\alpha = 0$ *to* $p$ **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha + (q_k)^\alpha$ ;
    $H_2.pop()$ ;
  **while** ***not*** $H_1.empty()$ ***and*** $H_1.top().thresh \leq t$ **do**
    $k \leftarrow H_1.top().\text{idx}$ ;
    **for** $\alpha = 0$ *to* $p$ **do**
      $\mathcal{M}_\alpha \leftarrow \mathcal{M}_\alpha - (q_k)^\alpha$ ; // $H_1 \leftarrow H_1 \setminus \{k\}$
    $H_1.pop()$ ;
  $\Delta_Q \leftarrow \sigma_D + f_p(\mathcal{M}_p, \mathcal{M}_{p-1}, \ldots, \mathcal{M}_1, |H_1|, t)$ ;
  $LB_i \leftarrow \Delta_Q \times G_U$ ;
$idx \leftarrow \arg\max_{1 \leq i \leq |\mathcal{T}|} LB_i$ ;
**return** $\mathcal{T}_{idx}$ ; // $t_L$ can be symmetrically obtained.

respectively.[5] However, adapting FSMELD to normalized sequences introduces a critical challenge: a single element $T[i]$ in the target series can assume different normalized values depending on the enclosing length-$m$ sliding window. Consequently, the normalized local envelope values associated with this position vary across candidate windows.

To guarantee the lower-bound property (Theorem 1 and the LCSS-specific bound in Section 3.5) under z-normalization, we must conservatively estimate the boundaries of the normalized envelopes. For any position $i$ within a local segment of length $\ell$, let $\mu_i^{\min}$ and $\mu_i^{\max}$ denote the minimum and maximum local means, and $\sigma_i^{\min}$ and $\sigma_i^{\max}$ the minimum and maximum local standard deviations, over all non-degenerate length-$m$ subsequences containing $i$. Positions covered only by degenerate windows are handled separately. These localized extreme values can be efficiently computed in amortized $O(1)$ time per target position via a monotonic queue mechanism, ensuring that the construction of local envelopes preserves the amortized sublinear time efficiency of the entire matching framework.

We define the conservative upper and lower envelope bounds $\hat{\mathbb{U}}_i^{\max}$ and $\hat{\mathbb{L}}_i^{\min}$ as follows:

$$\hat{\mathbb{U}}_i^{\max} = \max\left\{\frac{\mathbb{U}_i^T - \mu_i^{\min}}{\sigma_i^{\min}}, \frac{\mathbb{U}_i^T - \mu_i^{\min}}{\sigma_i^{\max}}\right\} \tag{33}$$

$$\hat{\mathbb{L}}_i^{\min} = \min\left\{\frac{\mathbb{L}_i^T - \mu_i^{\max}}{\sigma_i^{\min}}, \frac{\mathbb{L}_i^T - \mu_i^{\max}}{\sigma_i^{\max}}\right\} \tag{34}$$

For any non-degenerate candidate $X = T_{(k)}$ and relative position $r$, these bounds satisfy $\mathbb{U}_r^{\hat{X}} \leq \hat{\mathbb{U}}_{k+r-1}^{\max}$ and $\mathbb{L}_r^{\hat{X}} \geq \hat{\mathbb{L}}_{k+r-1}^{\min}$, preserving the lower-bound property.

Using these envelope bounds, we redefine the valley mask functions for normalized matching. The normalized indicator functions $\hat{g}_U$ and $\hat{g}_L$ are formally expressed as:

$$\hat{g}_U(i,t) = \mathbf{1}_{\{t \geq \hat{\mathbb{U}}_i^{\max}\}}, \quad \hat{g}_L(i,t) = \mathbf{1}_{\{t \leq \hat{\mathbb{L}}_i^{\min}\}}. \tag{35}$$

With these definitions in place, the normalized masks $\hat{g}_U$ and $\hat{g}_L$ replace the raw masks $g_U$ and $g_L$ in the sliding inner-product formulation. This robust adaptation ensures that our amortized $O(\log m)$ lower-bound evaluation cost is preserved even when rigorous scale and baseline invariance are enforced.

Unlike the target series, the query $Q$ is z-normalized only once, keeping its elements $q_i$ fixed throughout the search. Consequently, the penalty functions $h_U$ and $h_L$ are constructed exclusively from these static $q_i$ values and the sweeping threshold $t$. All volatility introduced by the sliding window's shifting mean and variance is absorbed entirely by the target-side mask functions ($\hat{g}_U$ and $\hat{g}_L$). This isolation decouples sequence morphology from amplitude scaling.

## 5.2 The Cascaded FSMELD Pipeline

To optimize the tightness and pruning power of our lower bound ($LB_{IP}$), we leverage the heuristic threshold optimization technique detailed in Section 4.4 to compute proper "water level" thresholds ($t_U$, $t_L$). A single global threshold for a massive time series $S$ often fails to capture local morphological variations, resulting in a loose lower bound; a strategic segmentation approach is therefore essential.

Rather than processing the entire series $S$ at once, we divide it into overlapping macro-segments (denoted by superscripts, e.g., $T^{(i)}$). This segmentation strategy serves a dual purpose. First, it allows for the calculation of locally optimized thresholds $(t_U, t_L)$ based on the conservative envelope bounds, enabling the lower bound to tightly adapt to local baseline volatility. Second, because continuous lower-bound evaluation is mathematically transformed into a sliding inner product using the aforementioned masks, it can be substantially accelerated via FFT. We set the segment length to $\ell = 2^{\lceil \log_2(8m) \rceil}$, using an empirical power-of-two FFT size while

[5] Subsequences with zero or near-zero standard deviation are treated as degenerate cases and handled separately to avoid division by zero and numerical instability.

keeping $\ell = \Theta(m)$. Each segment therefore contains $\Theta(m)$ candidate windows over which the costs of threshold selection and sliding-inner-product evaluation are amortized.

Algorithm 3 solves the fixed-cutoff range-search problem in Definition 3 and presents the complete FSMELD framework. To prevent false dismissals while minimizing exact-distance computations, these sublinear lower bounds are deployed within a cascaded filter. For numerical safety against FFT roundoff noise, our implementation evaluates pruning entirely in the raw cost space as $\max(0, lb_j + lb'_j) > \text{trans}^{-1}(\xi) + \delta_{\text{raw}}$, with a tolerance $\delta_{\text{raw}} = 10^{-5}$.[6] The $\max(0, \cdot)$ clamping prevents NaN exceptions during square-root transformations (e.g., ERP, DTW) and naturally preserves exact distance ties. Additionally, degenerate candidate windows (zero variance) bypass the FFT filter directly to exact verification. Candidates that survive this initial, fast filtering stage are then verified by progressively tighter (but more expensive) bounds, such as *BGLB*, before a final, exact elastic distance (*ELD*) computation is invoked. This multi-stage pruning architecture forms the core of the FSMELD framework. For simplicity, Algorithm 3 omits these low-level safeguards and standard tail handling.

**Algorithm 3:** FSMELD_normalized($S, Q, \xi$)

**Input:** Time series $S \in \mathbb{R}^n$, z-normalized query $Q \in \mathbb{R}^m$ and threshold $\xi \geq 0$.
**Output:** $\{i \mid ELD(\hat{S}_{(i)}, Q) \leq \xi\}$.

$\mathbb{A} \leftarrow \emptyset$ ;
$\ell \leftarrow 2^{\lceil \log_2(8m) \rceil}$ ;
$n_{seg} \leftarrow \lceil \frac{n-m+1}{\ell-m+1} \rceil$; $N_w \leftarrow \ell - m + 1$ ;
**foreach** $i \in \{0, 1, \ldots, n_{seg} - 1\}$ **do**
  $T^{(i)} \leftarrow (S[i \times N_w + 1], S[i \times N_w + 2], \ldots, S[i \times N_w + \ell])$ ;
  $\hat{\mathbb{U}}^{\max}, \hat{\mathbb{L}}^{\min} \leftarrow$ computed from $T^{(i)}$ using Eqs. (33) and (34) ;
  $t_U \leftarrow$ OptUpperThreshold($\hat{\mathbb{U}}^{\max}, Q$) ; // Alg. 2
  $t_L \leftarrow$ OptLowerThreshold($\hat{\mathbb{L}}^{\min}, Q$) ;
  $A \leftarrow (\hat{g}_U(1, t_U), \hat{g}_U(2, t_U), \ldots, \hat{g}_U(\ell, t_U))$ ;
  $B \leftarrow (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U))$ ;
  $\{lb_j\}_{j=1}^{N_w} \leftarrow \text{SIP}(A, B)$ ; // Alg. 1
  $A' \leftarrow (\hat{g}_L(1, t_L), \hat{g}_L(2, t_L), \ldots, \hat{g}_L(\ell, t_L))$ ;
  $B' \leftarrow (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L))$ ;
  $\{lb'_j\}_{j=1}^{N_w} \leftarrow \text{SIP}(A', B')$ ;
  **foreach** $j \in \{1, 2, \ldots, N_w\}$ **do**
    **if** $\xi \geq \text{trans}(lb_j + lb'_j) - \delta_{\text{FFT}}$ **then**
      **if** $\xi \geq BGLB(\hat{S}_{(j+i\times N_w)}, Q)$ **then**
        **if** $\xi \geq ELD(\hat{S}_{(j+i\times N_w)}, Q)$ **then**
          $\mathbb{A} \leftarrow \mathbb{A} \cup \{j + i \times N_w\}$ ;
**return** $\mathbb{A}$;

# 6 EXPERIMENTS

To rigorously assess the performance of the proposed FSMELD framework, we conduct extensive experiments aiming to answer the following research questions (RQs):

[6]For LCSS, the accumulated penalty $z$ counts certified lost matches; hence $\text{trans}(z) = z/m$, consistent with Eq. (6).

**Table 4: Datasets used in the experiments.**

| Dataset | Length $n$ | Description |
|---|---|---|
| ECG[6] | $1.7 \times 10^9$ | ECG signals from 15 subjects (1 kHz). |
| TEMP[7] | $1.9 \times 10^9$ | Monthly temperature records from China. |
| THCHS[8] | $1.2 \times 10^9$ | Mandarin speech from THCHS-30 (16 kHz). |
| PLA[9] | $9.0 \times 10^8$ | PLAID appliance current and voltage (30 kHz). |
| PLETH[10] | $4.2 \times 10^8$ | VTaC plethysmography (PPG, 250 Hz). |
| RW | $1.0 \times 10^9$ | Synthetic random walk. |

- **RQ1.** How effective is the proposed lower bound ($LB_{IP}$) in terms of pruning rate and lower-bound evaluation cost? (Section 6.2)
- **RQ2.** How does FSMELD compare to state-of-the-art exact subsequence search pipelines in end-to-end efficiency? (Section 6.3)
- **RQ3.** What are the specific contributions of the cascaded filters and the heuristic threshold optimization to the overall performance? (Section 6.4)
- **RQ4.** How robust is FSMELD to core parameter variations, such as the temporal constraint $w$ and the search cutoff $\xi$? (Section 6.5)

## 6.1 Experimental Setup

**Hardware.** All algorithms are implemented in C++ and evaluated on a workstation equipped with a 2.00 GHz Intel Core Ultra 7 255H CPU, 32 GB of RAM, and a 1-TB SSD.

**Datasets.** We employ a diverse collection of datasets to comprehensively evaluate our algorithms. This includes five real-world time series spanning various domains and one synthetic random walk dataset. The detailed characteristics of these datasets are summarized in Table 4.

**Algorithms.** To evaluate the proposed framework across four structurally diverse elastic measures (ERP, EDR, LCSS, and MSM), we carefully configure our baselines according to the specific evaluation task. For standalone lower-bound evaluations, we compare $LB_{IP}$ against GLB [19] and metric-specific bounds, including $LB_{\text{Keogh-ERP}}$ [7] for ERP, $LB_{\text{LCSS}}$ [25] for LCSS, and $LB_{\text{MSM}}$ [24] for MSM. For the end-to-end exact subsequence search, FSMELD is directly compared against the state-of-the-art generalized baseline, GLB+ELD. To ensure a strictly fair comparison, all surviving candidates across different filter pipelines are verified using the exact same self-implemented C++ band-constrained dynamic-programming verifier. Furthermore, cutoff-based early abandoning and DP-state pruning are disabled in the exact verification stage for all methods to isolate the filtering effectiveness of the lower bounds.

[6]https://physionet.org/content/butqdb/1.0.0/
[7]https://www.tpdc.ac.cn/zh-hans/data/71ab4677-b66c-4fd1-a004-b2a541c4d5bf
[8]https://www.openslr.org/18/
[9]https://figshare.com/articles/dataset/PLAID_-_A_Voltage_and_Current_Measurement_Dataset_for_Plug_Load_Appliance_Identification_in_Households/10084619
[10]https://physionet.org/content/vtac/1.0/

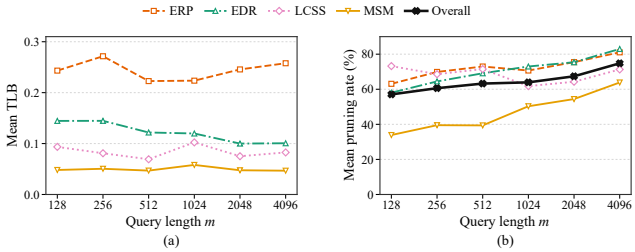



**Figure 3: Effectiveness of $LB_{IP}$ under $w = \lfloor 0.05m \rfloor$: (a) mean TLB and (b) mean pruning rate over the six datasets.**

**Parameters and protocol.** We use $g = 0$ for ERP, $\epsilon = 0.1$ for EDR, $\epsilon = 0.2$ for LCSS, and $c = 0.5$ for MSM. Unless stated otherwise, $m \in \{128, 256, 512, 1024, 2048, 4096\}$, $w = \lfloor 0.05m \rfloor$, and $\ell = 2^{\lceil \log_2(8m) \rceil}$.

For each configuration, all methods use the same z-normalized query and candidate windows, temporal constraint, and cutoff. The cutoff $\xi$ is calibrated offline as the tenth-smallest exact distance under the corresponding measure and constraint and remains fixed throughout the range search. Because the task is range search rather than truncated top-$k$ retrieval, distance ties at the cutoff may yield more than ten results; all valid candidates with exact distance at most $\xi$, up to a numerical tolerance of $10^{-9}$, are retained.

## 6.2 Lower-Bound Effectiveness

We first evaluate the theoretical efficacy of $LB_{IP}$ independent of the exact verification stage.

**Tightness and Pruning Power.** We measure tightness as $TLB = LB_{IP}(X, Q)/ELD(X, Q)$, where $TLB \leq 1$ and larger values indicate tighter bounds. As shown in Figure 3(a), the tightness remains highly stable across varying query lengths. Furthermore, Figure 3(b) reports the pruning rate (i.e., the fraction of candidates safely discarded). Under $w = \lfloor 0.05m \rfloor$, $LB_{IP}$ achieves a robust mean pruning rate of 64.5% across all six datasets. Notably, the pruning power scales positively with query length, increasing from 57.1% at $m = 128$ to 74.8% at $m = 4096$, demonstrating its exceptional filtering capability for long queries.

**Lower-Bound Evaluation Cost.** To demonstrate the computational advantage of our amortized sublinear approach, we compare the online evaluation time of $LB_{IP}$ against GLB and three metric-specific bounds ($LB_{\text{Keogh-ERP}}$ for ERP, $LB_{\text{LCSS}}$ for LCSS, and $LB_{\text{MSM}}$ for MSM). The timing includes all necessary steps for lower-bound computation per candidate (e.g., sliding statistics, normalization, envelope/mask construction, and SIP computation). As illustrated in Figure 4, the average cost of $LB_{IP}$ remains low and stable, whereas the costs of competing $O(m)$ bounds increase linearly. Overall, $LB_{IP}$ achieves geometric-mean evaluation speedups of 45.3× over GLB and 14.3× over the metric-specific bounds.

## 6.3 End-to-End Search Efficiency

We next evaluate whether the theoretical speedups of $LB_{IP}$ translate into practical end-to-end improvements in exact subsequence range search. The runtime encompasses data loading, online preprocessing, all filtering stages, and exact verification.

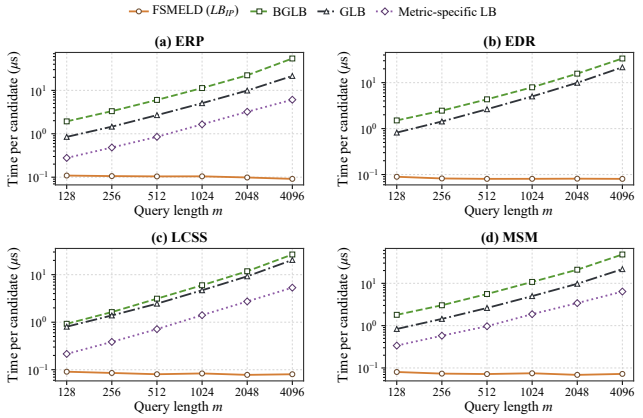



**Figure 4: Mean online lower-bound evaluation time per candidate under $w = \lfloor 0.05m \rfloor$, averaged over the six datasets.**

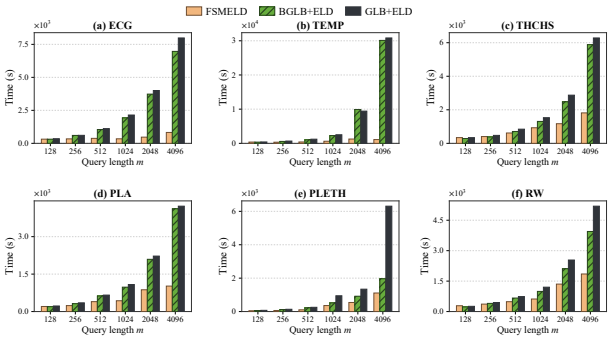



**Figure 5: End-to-end performance of FSMELD, BGLB+ELD, and GLB+ELD for different query lengths under $w = \lfloor 0.05m \rfloor$.**

**Comparison against GLB+ELD:** As illustrated in Figure 5, FSMELD significantly outperforms the GLB+ELD pipeline. Specifically, FSMELD achieves an overall geometric-mean speedup of 2.64× over GLB+ELD. This advantage is highly consistent across all four individual elastic measures, with per-measure speedups ranging from 1.89× to 3.78×. Furthermore, the performance gap widens as the query length $m$ increases. This trend supports our theoretical claim that the amortized sublinear evaluation of $LB_{IP}$ becomes increasingly beneficial for long sequence queries compared to traditional linear-time filters.

## 6.4 Ablation Study

**Contribution of the Cascaded Filters.** FSMELD combines the fast $LB_{IP}$ pre-filter with a tighter linear-time filter (i.e., BGLB) before the exact elastic verification. To examine the role of each component, we consider the complete pipeline ($LB_{IP}$+BGLB+ELD) alongside two ablated variants: BGLB+ELD and $LB_{IP}$+ELD. Removing $LB_{IP}$ sends all candidates directly to the $O(m)$ BGLB evaluation, whereas removing BGLB sends all first-stage survivors directly to the exact distance verification.

**Table 5: Cascade ablation under $w = \lfloor 0.05m \rfloor$. Runtime (s) is the arithmetic mean over ECG and RW and the four elastic measures.**

| | Query length $m$ | | |
|---|---|---|---|
| **Variant** | 256 | 512 | 1024 |
| Remove $LB_{IP}$ | 528.6 | 907.6 | 1612.7 |
| Remove BGLB | 13976.6 | 43683.6 | 139555.3 |
| **FSMELD** | **352.9** | **431.8** | **480.9** |

**Table 6: Comparison of exact and heuristic threshold optimization on ECG under $w = \lfloor 0.05m \rfloor$, averaged over ERP, EDR, LCSS, and MSM.**

| | Pruning Rate (%) | | Speedup over GLB+ELD (×) | |
|---|---|---|---|---|
| $m$ | Exact | Heuristic | Exact | Heuristic |
| 256 | 76.73 | 76.71 | 0.68 | 1.89 |
| 512 | 82.32 | 82.37 | 0.79 | 2.96 |
| 1024 | 92.05 | 92.08 | 1.18 | 6.01 |
| 2048 | 92.63 | 92.66 | 1.50 | 9.11 |

The experiment uses ECG and RW datasets with $m \in \{256, 512, 1024\}$ and $w = \lfloor 0.05m \rfloor$. Table 5 reports the arithmetic mean runtime over both datasets and the four elastic measures. The mean runtimes of BGLB+ELD are 1.50×, 2.10×, and 3.35× those of FSMELD at the three query lengths, respectively. Furthermore, the high runtime of $LB_{IP}$+ELD demonstrates that while $LB_{IP}$ evaluates in amortized sublinear time, a tighter secondary filter is necessary to prevent the exact dynamic-programming verifier from becoming a bottleneck. This supports our cascaded architecture: $LB_{IP}$ prunes the majority of unpromising windows, minimizing the workload for the subsequent linear-time filter.

**Impact of Heuristic Threshold Optimization.** To understand the design trade-offs of the $LB_{IP}$ framework, we evaluate the heuristic threshold optimization introduced in Section 4. We compare the exact objective $p_1(t) + p_2(t) + p_3(t)$ with the heuristic objective $p_1(t)$. Both strategies use identical queries, cutoffs, secondary filtering, and exact-verification stages. Figure 6 illustrates the corresponding upper-threshold objective landscapes. For each measure, we sample 1000 valid ECG macro-segments with $m = 1024$ and $w = \lfloor 0.05m \rfloor$. The heuristic objective tracks the high-value region of the full objective across all four measures, and the selected thresholds coincide in the representative cases shown.

Table 6 reports the quantitative results on ECG under $w = \lfloor 0.05m \rfloor$, averaged over ERP, EDR, LCSS, and MSM. The heuristic strategy achieves pruning rates nearly identical to those of the exact optimization, with the difference remaining within 0.1 percentage points. However, by avoiding the computation of boundary terms $p_2(t)$ and $p_3(t)$, the heuristic strategy lowers the computational overhead of threshold optimization. Consequently, the geometric-mean end-to-end speedup over GLB+ELD increases from 0.99× (with exact optimization) to 4.18× (with heuristic optimization), demonstrating that omitting the boundary terms provides a better cost-pruning trade-off.

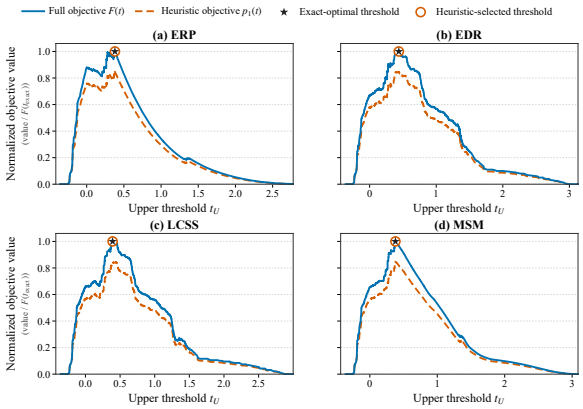



**Figure 6: Full and heuristic upper-threshold objectives for representative ECG macro-segments.**

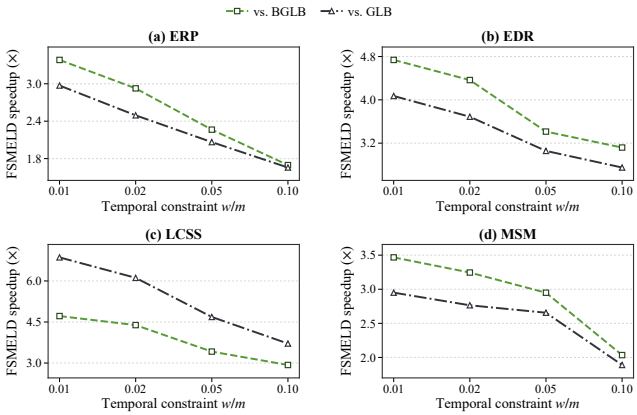



**Figure 7: Impact of the temporal constraint $w$ on the end-to-end speedup of FSMELD.**

## 6.5 Parameter Sensitivity

**Sensitivity to Temporal Constraint $w$.** To evaluate the impact of the temporal constraint $w$, we fix $m = 1024$ and vary the ratio $w/m \in \{0.01, 0.02, 0.05, 0.10\}$ on the ECG and RW datasets across all four elastic measures. For each configuration, the cutoff is recalibrated to the tenth-smallest exact distance. As shown in Figure 7, FSMELD consistently outperforms the GLB+ELD pipeline across the entire tested range. While the relative speedup gradually decreases as a larger $w$ relaxes the warping constraint, FSMELD maintains a clear efficiency advantage.

**Sensitivity to Search Cutoff $\xi$.** We assess cutoff sensitivity using fixed excerpts of $K = 10^7$ candidate windows from the ECG and RW datasets, setting $m = 1024$ and $w = 51$. The cutoff $\xi$ is dynamically defined as the $\lceil qK \rceil$-th smallest exact distance, where $q \in \{0.0001\%, 0.001\%, 0.01\%, 0.1\%, 1\%\}$ denotes the nominal cutoff-rank fraction. The actual fraction of returned candidates may exceed $q$ because of distance ties.

As illustrated in Figure 8(a), increasing $q$ naturally reduces the pruning rates for all bounds and increases the overall runtimes

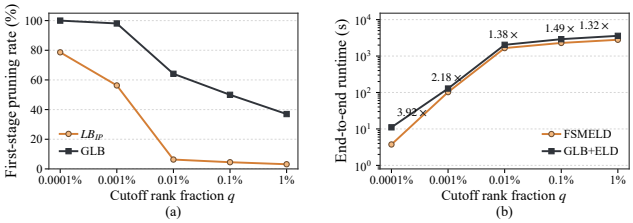



**Figure 8: Sensitivity to the cutoff rank fraction $q$. (a) Pruning rates of $LB_{IP}$ and GLB. (b) End-to-end runtimes on a logarithmic scale. Curves show arithmetic means over ECG and RW and the four elastic measures.**

(Figure 8(b)). Although GLB achieves higher absolute pruning rates than $LB_{IP}$ at looser cutoffs, FSMELD consistently delivers lower end-to-end runtimes than GLB+ELD at all five cutoff levels. The geometric-mean speedup ranges from 1.32× to 3.92×, with the maximum efficiency gain observed at the strictest cutoff ($q = 0.0001\%$). This confirms that FSMELD excels in selective top-$k$ retrieval scenarios while remaining strictly superior even under relaxed cutoffs.

## 7 CONCLUSION

In this paper, we introduced FSMELD, an exact filter–verify framework for subsequence matching under DTW, ERP, EDR, LCSS, and MSM. Its $LB_{IP}$ formulation decomposes envelope penalties into sliding inner products, enabling FFT-based evaluation in amortized $O(\log m)$ time per candidate. Experiments demonstrate that FSMELD achieves high pruning rates and outperforms state-of-the-art exact baselines, providing a practical framework for large-scale time series analytics.

## A CORRECTNESS OF THE LCSS INSTANTIATION

For equal-length sequences $X, Q \in \mathbb{R}^m$, let $\epsilon \geq 0$ be the LCSS matching tolerance and let $t_U, t_L \in \mathbb{R}$ be arbitrary thresholds. We use the masks and binary penalties of the LCSS instantiation in the main paper.

Consider the upper-envelope case. If $\mathbb{U}_i^X \leq t_U$ and $q_i > t_U + \epsilon$, then

$$q_i > t_U + \epsilon \geq \mathbb{U}_i^X + \epsilon. \tag{36}$$

According to the temporal constraint of LCSS, all temporally admissible target elements associated with the $i$-th query position are bounded above by $\mathbb{U}_i^X$. Therefore, $g_U(i, t_U)h_U(i, t_U) = 1$ certifies that the corresponding query position cannot participate in an admissible LCSS match. The lower-envelope case follows symmetrically.

Let the total number of certified lost matches be

$$\begin{aligned} K_{\mathrm{LCSS}} = &\sum_{i=1}^{m} g_U(i, t_U)h_U(i, t_U) \\ &+ \sum_{i=1}^{m} g_L(i, t_L)h_L(i, t_L). \end{aligned} \tag{37}$$

Because $\mathbb{U}_i^X \geq \mathbb{L}_i^X$ and $\epsilon \geq 0$, the upper- and lower-envelope conditions cannot simultaneously certify the same query position as a lost match: the former requires $q_i > \mathbb{U}_i^X + \epsilon$, whereas the latter requires $q_i < \mathbb{L}_i^X - \epsilon$. Hence, $K_{\mathrm{LCSS}}$ counts distinct query positions that cannot be matched. Since an LCSS matching uses each query position at most once, we obtain

$$\mathrm{LCSS}(X, Q) \leq m - K_{\mathrm{LCSS}}. \tag{38}$$

Using the normalized LCSS distance, we obtain

$$\begin{aligned} d_{\mathrm{LCSS}}(X, Q) &= 1 - \frac{\mathrm{LCSS}(X, Q)}{m} \\ &\geq \frac{K_{\mathrm{LCSS}}}{m} = LB_{IP}(X, Q, t_U, t_L), \end{aligned} \tag{39}$$

where the last equality uses $\mathrm{trans}(z) = z/m$.

The same reasoning applies to any conservative envelopes that contain all temporally admissible target values, including the shared envelopes used for z-normalized candidates.

## B OPTIMIZATION OF $LB_{IP}$ FOR A SINGLE SUBSEQUENCE PAIR

Theorem 4. *Given a query sequence $Q \in \mathbb{R}^m$ and a specific candidate subsequence $X \in \mathbb{R}^m$, the exact thresholds $(t_U^*, t_L^*)$ that maximize the proposed inner-product lower bound $LB_{IP}(X, Q, t_U, t_L)$ can be computed in $O(m \log m)$ time.*

Proof. Without loss of generality, we focus on computing the optimal upper threshold $t_U^*$, since the derivation for $t_L^*$ follows symmetrically.

By the definition of the $LB_{IP}$ framework, the threshold $t_U$ independently dictates the penalty term associated with the upper envelope. Let $F_U(t_U)$ denote the threshold-dependent objective function to be maximized within $LB_{IP}$. For the established elastic distances, $F_U(t_U)$ can be formulated as an element-wise summation over the sequence length $m$:

$$F_U(t_U) = \sum_{i=1}^{m} \phi_i(t_U) \tag{40}$$

where $\phi_i(t_U) = g_U(i, t_U)h_U^{\mu}(i, t_U)$. Depending on the elastic measure $\mu$, $\phi_i$ may be piecewise constant, piecewise linear, or piecewise quadratic. Let $\mathbb{B}_U$ contain all envelope-activation values $\mathbb{U}_i^X$ and all metric-specific query-state transition values, such as $q_i$ and $\tau_\mu(q_i)$. Since each element contributes only a constant number of events, $|\mathbb{B}_U| = O(m)$.

By Theorem 2, while the target-side mask remains fixed, every active upper-side penalty is non-increasing as $t_U$ increases. Therefore, a maximizer can be selected from the envelope-activation set $\{\mathbb{U}_i^X\}_{i=1}^m$. The additional query-state events in $\mathbb{B}_U$ are used only to update the response-function statistics during the sweep.

A naive evaluation of $F_U(b)$ for each candidate $b \in \mathbb{B}_U$ requires $O(m^2)$ time. However, this process can be optimized to $O(m \log m)$ using a two-phase sweep-line algorithm:

(1) **Sorting Phase:** We extract all $O(m)$ breakpoints into the set $\mathbb{B}_U$ and sort them in ascending order. This step requires $O(m \log m)$ time.
(2) **Sweep-line Evaluation Phase:** We maintain metric-specific sufficient statistics while sweeping through the sorted events. DTW and ERP maintain zeroth-, first-, and second-order

moments; EDR and LCSS maintain counts; and MSM maintains counts and first-order moments for its linear and saturated states. Each query element changes state only a constant number of times, and each transition updates a constant number of statistics. Consequently, after sorting, all candidate envelope thresholds can be evaluated in $O(m)$ time.

Since the sweep-line evaluation processes the $O(m)$ breakpoints in $O(m)$ time, the overall computational complexity is dominated by the sorting phase, resulting in $O(m \log m)$. The identical algorithmic logic applies symmetrically to the derivation of $t_L^*$, thereby concluding the proof. □

# C EXACT THRESHOLD OPTIMIZATION FOR A TARGET SEGMENT

This appendix gives the complete algorithm underlying Theorem 3. We retain the notation and query-state representation of Section 4.4. The exact procedure maximizes $p_1(t) + p_2(t) + p_3(t)$, using the same two heaps and query moments as the heuristic, together with additional statistics for the boundary terms.

## C.1 Objective and Boundary Statistics

Let $\mathcal{U} \in \mathbb{R}^\ell$ be the shared upper envelope and $Q \in \mathbb{R}^m$ the query, with $\ell \geq 2m$. For the raw formulation, $\mathcal{U} = \mathbb{U}^T$, and $g_U(j,t) = \mathbf{1}_{\{\mathcal{U}_j \leq t\}}$. The functions $h_U$, $D$, and $\tau$, and the moment order $p$, are those used in Section 4.4.2. In particular, the LCSS convention $D(q_i) = 1$ denotes a certified lost match in the threshold-selection procedure.

Recall that $H_2$ stores query indices with $t < \tau(q_i)$, whereas $H_1$ stores those with $\tau(q_i) \leq t < q_i$. Indices with $q_i \leq t$ belong to neither heap. The unweighted statistics remain

$$\mathcal{M}_\alpha = \sum_{i \in H_1} q_i^\alpha, \qquad \sigma_D = \sum_{i \in H_2} D(q_i), \tag{41}$$

where $\mathcal{M}_0 = |H_1|$ and $\alpha = 0, \ldots, p$. They evaluate the total query penalty as

$$\Delta_Q(Q,t) = \sigma_D + f_p(\mathcal{M}_p, \ldots, \mathcal{M}_1, |H_1|, t). \tag{42}$$

As in the heuristic, $G_U$ counts active core positions $m \leq j \leq \ell - m + 1$, giving $p_1(t) = G_U \Delta_Q(Q,t)$.

For query position $i$, a target position $j$ participates in the cumulative objective precisely when $i \leq j \leq \ell - m + i$. Its associated boundary positions are therefore

$$\mathcal{B}_i = \{i, \ldots, m-1\} \cup \{\ell - m + 2, \ldots, \ell - m + i\}. \tag{43}$$

An integer range whose lower endpoint exceeds its upper endpoint is empty. Thus $|\mathcal{B}_i| = m - 1$, including the empty-boundary case $m = 1$. These sets are represented by their two index ranges; their elements need not be stored separately.

Reordering the boundary contributions in Eq. (26) gives

$$p_2(t) + p_3(t) = \sum_{i=1}^{m} h_U(i,t) \sum_{j \in \mathcal{B}_i} g_U(j,t). \tag{44}$$

We maintain the corresponding boundary-weighted statistics:

$$\begin{aligned} \mathcal{M}_\alpha^B &= \sum_{i \in H_1} q_i^\alpha \sum_{j \in \mathcal{B}_i} g_U(j,t), \\ \sigma_D^B &= \sum_{i \in H_2} D(q_i) \sum_{j \in \mathcal{B}_i} g_U(j,t). \end{aligned} \tag{45}$$

The superscript $B$ identifies boundary statistics. In particular, $\mathcal{M}_0^B$ counts active boundary contributions associated with indices in $H_1$; it is not the cardinality of $H_1$.

The same polynomial $f_p$ evaluates the boundary contribution:

$$\begin{aligned} p_2(t) + p_3(t) = \sigma_D^B \\ + f_p(\mathcal{M}_p^B, \ldots, \mathcal{M}_1^B, \mathcal{M}_0^B, t). \end{aligned} \tag{46}$$

This follows because $f_p$ is linear in its moment and count arguments. For example, its boundary-weighted value for ERP and DTW is $\mathcal{M}_2^B - 2t\mathcal{M}_1^B + t^2\mathcal{M}_0^B$. For EDR and LCSS, the contribution of $H_1$ remains zero: $f_0(\mathcal{M}_0^B, t) = 0$. Positive-order moment arguments are omitted when $p = 0$.

## C.2 Exact Threshold-Selection Procedure

Algorithm **??** constructs $\mathcal{T}$ from all distinct values of $\mathcal{U}$ and records the target positions associated with each value. It initializes the query states at $t_0 = \mathcal{T}_1$ as in Algorithm 2. Initially, $G_U$ and all boundary statistics are zero because no target masks have yet been activated.

At each candidate threshold $t$, the procedure first processes all query-state transitions up to $t$. At this stage, the active target positions are exactly those with $\mathcal{U}_j < t$. For a transitioning query index $k$, let

$$b = \sum_{j \in \mathcal{B}_k} \mathbf{1}_{\{\mathcal{U}_j < t\}}. \tag{47}$$

The value $b$ is computed by scanning the two ranges in Eq. (43), in $O(m)$ time. When $k$ moves from $H_2$ to $H_1$, its fixed contributions $D(q_k)$ and $bD(q_k)$ are removed from $\sigma_D$ and $\sigma_D^B$, respectively. Its moments are then added to $\mathcal{M}_\alpha$ and, with multiplicity $b$, to $\mathcal{M}_\alpha^B$. When $k$ leaves $H_1$, these moments are removed.

The procedure next activates all target positions with $\mathcal{U}_j = t$. A core position increments $G_U$. For a boundary position $j$, the associated query indices are

$$\max\{1, j - \ell + m\} \leq k \leq \min\{m, j\}. \tag{48}$$

For each such $k$, a fixed-penalty state contributes $D(q_k)$ to $\sigma_D^B$, a post-transition state contributes $q_k^\alpha$ to each $\mathcal{M}_\alpha^B$, and a submerged state contributes zero.

The strict inequality in Eq. (47) is essential: masks with $\mathcal{U}_j = t$ are added only in the subsequent activation phase. Query transitions use $\tau(q_k) \leq t$ and $q_k \leq t$, so the zero penalty at $t = q_k - \epsilon$ is handled correctly for EDR and LCSS. Query-only breakpoints need not be evaluated as candidates; the heaps process all transitions crossed before the next envelope threshold.

Here, $LB_i$ is the full cumulative raw-penalty objective at $t = \mathcal{T}_i$. The groups of target positions are stored alongside their sorted envelope values; retrieving a group does not require scanning $\mathcal{U}$ again.

## C.3 Correctness and Complexity

Theorem 5. *Algorithm ?? returns a threshold maximizing $p_1(t) + p_2(t) + p_3(t)$ for the supplied shared envelope and query. Its running time is $O(m^2 + \ell \log \ell)$ and its auxiliary space is $O(\ell + m)$.*

Proof. We show that after processing a candidate threshold $t$, the heaps represent the query states at $t$, $G_U$ counts the active core masks, and the boundary statistics satisfy Eq. (45).

At $t_0$, the query states and unweighted moments are initialized directly. There are no envelope values below $t_0$, so all boundary statistics are initially zero. Activating the positions with envelope value $t_0$ establishes the stated invariant.

For a subsequent candidate $t$, all previously active target positions have $\mathcal{U}_j < t$, and no other target positions have yet been activated. Thus the value $b$ in Eq. (47) gives exactly the multiplicity with which the transitioning query element contributes to the boundary terms. The two while loops update its unweighted and boundary-weighted contributions together. Since $\tau(q_k) \le q_k$, transferring elements from $H_2$ before removing elements from $H_1$ also handles a query that crosses both transitions between successive envelope thresholds.

The activation phase then adds each newly active core or boundary position exactly once, using the query states already updated to $t$. The invariant therefore holds after this phase. Equations (44) and (46) imply that $LB_i$ equals $p_1(t) + p_2(t) + p_3(t)$. By Theorem 2, evaluating all distinct envelope values includes an exact maximizer.

Sorting the envelope values and grouping their positions costs $O(\ell \log \ell)$ time. Each query element enters and leaves each heap at most once, giving $O(m \log m)$ heap maintenance. There are at most $2m$ query-state transitions that require a boundary scan, and each scan examines $m - 1$ positions, for $O(m^2)$ total work. During boundary activation, the total number of associated query positions visited is

$$\sum_{j=1}^{m-1} j + \sum_{j=\ell-m+2}^{\ell} (\ell - j + 1) = m(m-1).$$

Each visit updates at most $p+1$ moments, where $p \le 2$ is a measure-specific constant. Core activations and final objective evaluations cost $O(\ell)$ in total. Hence the overall running time is $O(m^2 + \ell \log \ell)$.

The sorted position groups and recorded objective values use $O(\ell)$ space, and the heaps use $O(m)$ space. The sets $\mathcal{B}_i$ are enumerated from their two index ranges without storing all boundary pairs. The auxiliary space is therefore $O(\ell + m)$. □

## C.4 Computing the Lower Threshold

The lower threshold can be obtained by reflecting the input values and reusing the same procedure. Given the shared lower envelope $\mathcal{L}$, compute

$$t_L^* = -\textsc{ExactUpperThreshold}(-\mathcal{L}, -Q). \tag{49}$$

For ERP, the reflected call also replaces the gap value $g$ by $-g$; the parameters $\epsilon$ and $c$ remain unchanged for EDR, LCSS, and MSM. Under this reflection, $\mathcal{L}_j \ge t$ is equivalent to $-\mathcal{L}_j \le -t$, and each lower-side penalty equals the corresponding upper-side penalty for the reflected query. The returned threshold therefore maximizes the cumulative lower penalty. Computing both thresholds retains the $O(m^2 + \ell \log \ell)$ time bound.

# D FSMELD FOR RAW SUBSEQUENCE MATCHING

While the main text of this paper focuses on subsequence matching under z-score normalization—the standard prerequisite for handling baseline and scale variations in time series queries—the FSMELD framework is natively applicable to raw subsequence matching. In this appendix, we detail the algorithm when executed directly on unnormalized time series data.

Compared to the normalized version, raw subsequence matching completely bypasses the dynamic bounding estimations for local means and standard deviations. Consequently, the segment upper and lower envelopes ($\mathbb{U}^{(i)}$ and $\mathbb{L}^{(i)}$) can be utilized directly as static boundaries, making the $LB_{IP}$ formulation even more straightforward. The complete procedure is presented in Algorithm 5.

*Strategic Macro-Segmentation and Envelopes.* Because a single global threshold for a massive time series $S \in \mathbb{R}^n$ fails to capture local morphological variations, FSMELD divides the long target series $S$ into overlapping macro-segments. As shown in Algorithm 5 (Lines 2-3), the segment length is defined as $\ell = 2^{\lceil \log_2(8m) \rceil}$, which utilizes a power-of-two size to optimize the internal Fast Fourier Transform (FFT) operations while strictly keeping $\ell = \Theta(m)$. The total number of segments is $n_{seg} = \lceil \frac{n}{\ell - m + 1} \rceil$, with adjacent segments overlapping by $m - 1$ points to ensure no candidate windows are missed at the boundaries. For each segment $T^{(i)}$, we directly compute its raw upper and lower envelopes $\mathbb{U}^{(i)}$ and $\mathbb{L}^{(i)}$ based on the temporal constraint $w$.

*Threshold Optimization and Vector Construction.* Because raw sequence matching does not involve shifting means and variances, the heuristic threshold optimization procedures (OptUpperThreshold and OptLowerThreshold) are directly applied to the raw envelopes (Lines 7-8). Once the locally optimal thresholds $t_U$ and $t_L$ are selected for the current segment, the target series is transformed into binary valley mask arrays ($A$ and $A'$), and the query sequence $Q$ is transformed into numeric protrusion penalty arrays ($B$ and $B'$).

*Amortized Lower Bound Evaluation via FFT..* With the morphological alignment decoupled into the aforementioned vector representations, evaluating the lower bound across all $N_w = \ell - m + 1$ overlapping candidate subsequences within the segment mathematically equates to calculating the discrete linear convolutions. We encapsulate this operation into the Fast Sliding Inner Product (SIP) function (Lines 11 and 14). By leveraging the Convolution Theorem, the SIP function computes all inner products for the segment in $O(\ell \log \ell)$ time. Since each segment evaluates $\Theta(m)$ candidates simultaneously, the amortized cost of this lower bound evaluation is rigorously reduced to $O(\log m)$ per candidate window.

*Cascaded Verification Pipeline.* To guarantee no false dismissals while strictly minimizing the number of exact dynamic-programming computations, FSMELD relies on a cascaded filtering architecture (Lines 15-21). For any candidate subsequence $S_{(j+i\times N_w)}$, its inner-product lower bound is first checked against the cutoff $\xi$. To ensure numerical safety, this pruning comparison is offset by a conservative tolerance $\delta_{\text{FFT}}$, which accounts for floating-point roundoff

errors introduced during the FFT convolutions. Candidates surviving this initial $LB_{IP}$ filter are then verified by a tighter, linear-time bound (i.e., *BGLB*). Only the minimal subset of candidates that survive all lower-bounding stages will trigger the exact elastic distance evaluation (*ELD*).

## REFERENCES


[1] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *Knowledge Discovery in Databases: Papers from the 1994 AAAI Workshop (AAAI Technical Report)*, Usama M. Fayyad and Ramasamy Uthurusamy (Eds.). AAAI Press, Menlo Park, CA, USA, 359–370. https://aaai.org/papers/359-ws94-03-031/

[2] Karl Bringmann and Marvin Künnemann. 2015. Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping. In *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, Piscataway, NJ, USA, 79–97. https://doi.org/10.1109/FOCS.2015.15

[3] Zemin Chao, Hong Gao, Yinan An, and Jianzhong Li. 2022. The Inherent Time Complexity and an Efficient Algorithm for Subsequence Matching Problem. *Proceedings of the VLDB Endowment* 15, 7 (2022), 1453–1465. https://doi.org/10.14778/3523210.3523222

[4] Zemin Chao, Hong Gao, Dongjing Miao, Jianzhong Li, and Hongzhi Wang. 2025. An Amortized $O(1)$ Lower Bound for Dynamic Time Warping in Motif Discovery. *IEEE Transactions on Knowledge and Data Engineering* 37, 5 (2025), 2239–2252. https://doi.org/10.1109/TKDE.2025.3544751

[5] Zemin Chao, Boyu Xiao, Zitong Li, Zhixin Qi, Xianglong Liu, and Hongzhi Wang. 2026. A New Lower Bounding Paradigm and Tighter Lower Bounds for Elastic Similarity Measures. arXiv preprint arXiv:2603.14899. https://doi.org/10.48550/arXiv.2603.14899 arXiv:2603.14899 [cs.DB]

[6] Zemin Chao, Qiaoyi Zheng, Zhixin Qi, and Hongzhi Wang. 2025. FSMDTW: A Fast Index-Free Subsequence Matching Algorithm for Dynamic Time Warping. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3628–3640. https://doi.org/10.14778/3748191.3748220

[7] Lei Chen and Raymond Ng. 2004. On the Marriage of $L_p$-Norms and Edit Distance. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases*. Morgan Kaufmann, San Francisco, CA, USA, 792–803. https://doi.org/10.1016/B978-012088469-8.50070-X

[8] Lei Chen, M. Tamer Özsu, and Vincent Oria. 2005. Robust and Fast Similarity Search for Moving Object Trajectories. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 491–502. https://doi.org/10.1145/1066157.1066213

[9] Jens E. d'Hondt, Haojun Li, Fan Yang, Odysseas Papapetrou, and John Paparrizos. 2025. A Structured Study of Multivariate Time-Series Distance Measures. *Proceedings of the ACM on Management of Data* 3, 3, Article 121 (2025), 29 pages. https://doi.org/10.1145/3725258

[10] Hui Ding, Goce Trajcevski, Peter Scheuermann, Xiaoyue Wang, and Eamonn J. Keogh. 2008. Querying and Mining of Time Series Data: Experimental Comparison of Representations and Distance Measures. *Proceedings of the VLDB Endowment* 1, 2 (2008), 1542–1552. https://doi.org/10.14778/1454159.1454226

[11] Christos Faloutsos, M. Ranganathan, and Yannis Manolopoulos. 1994. Fast Subsequence Matching in Time-Series Databases. *ACM SIGMOD Record* 23, 2 (1994), 419–429. https://doi.org/10.1145/191843.191925

[12] Matthieu Herrmann and Geoffrey I. Webb. 2021. Early Abandoning and Pruning for Elastic Distances Including Dynamic Time Warping. *Data Mining and Knowledge Discovery* 35, 6 (2021), 2577–2601. https://doi.org/10.1007/s10618-021-00782-4

[13] Christopher Holder, Matthew Middlehurst, and Anthony Bagnall. 2024. A Review and Evaluation of Elastic Distance Functions for Time Series Clustering. *Knowledge and Information Systems* 66, 2 (2024), 765–809. https://doi.org/10.1007/s10115-023-01952-0

[14] Eamonn Keogh and Shruti Kasetty. 2002. On the Need for Time Series Data Mining Benchmarks: A Survey and Empirical Demonstration. In *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 102–111. https://doi.org/10.1145/775047.775062

[15] Eamonn Keogh and Chotirat Ann Ratanamahatana. 2005. Exact Indexing of Dynamic Time Warping. *Knowledge and Information Systems* 7, 3 (2005), 358–386. https://doi.org/10.1007/s10115-004-0154-9

[16] Sang-Wook Kim, Sanghyun Park, and Wesley W. Chu. 2001. An Index-Based Approach for Similarity Search Supporting Time Warping in Large Sequence Databases. In *Proceedings of the 17th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 607–614. https://doi.org/10.1109/ICDE.2001.914875

[17] Daniel Lemire. 2009. Faster Retrieval with a Two-Pass Dynamic-Time-Warping Lower Bound. *Pattern Recognition* 42, 9 (2009), 2169–2180. https://doi.org/10.1016/j.patcog.2008.11.030

[18] Jason Lines and Anthony Bagnall. 2015. Time Series Classification with Ensembles of Elastic Distance Measures. *Data Mining and Knowledge Discovery* 29, 3 (2015), 565–592. https://doi.org/10.1007/s10618-014-0361-2

[19] John Paparrizos, Kaize Wu, Aaron J. Elmore, Christos Faloutsos, and Michael J. Franklin. 2023. Accelerating Similarity Search for Elastic Measures: A Study and New Generalization of Lower Bounding Distances. *Proceedings of the VLDB Endowment* 16, 8 (2023), 2019–2032. https://doi.org/10.14778/3594512.3594530

[20] Thanawin Rakthanmanon, Bilson Campana, Abdullah Mueen, Gustavo Batista, Brandon Westover, Qiang Zhu, Jesin Zakaria, and Eamonn Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 262–270. https://doi.org/10.1145/2339530.2339576

[21] Ahmed Shifaz, Charlotte Pelletier, François Petitjean, and Geoffrey I. Webb. 2023. Elastic Similarity and Distance Measures for Multivariate Time Series. *Knowledge and Information Systems* 65, 6 (2023), 2665–2698. https://doi.org/10.1007/s10115-023-01835-4

[22] Alexandra Stefan, Vassilis Athitsos, and Gautam Das. 2012. The Move-Split-Merge Metric for Time Series. *IEEE Transactions on Knowledge and Data Engineering* 25, 6 (2012), 1425–1438. https://doi.org/10.1109/TKDE.2012.88

[23] Chang Wei Tan, François Petitjean, and Geoffrey I. Webb. 2019. Elastic Bands across the Path: A New Framework and Method to Lower Bound DTW. In *Proceedings of the 2019 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 522–530. https://doi.org/10.1137/1.9781611975673.59

[24] Chang Wei Tan, François Petitjean, and Geoffrey I. Webb. 2020. FastEE: Fast Ensembles of Elastic Distances for Time Series Classification. *Data Mining and Knowledge Discovery* 34, 1 (2020), 231–272. https://doi.org/10.1007/s10618-019-00663-x

[25] Michail Vlachos, Marios Hadjieleftheriou, Dimitrios Gunopulos, and Eamonn Keogh. 2003. Indexing Multi-Dimensional Time-Series with Support for Multiple Distance Measures. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, New York, NY, USA, 216–225. https://doi.org/10.1145/956750.956777

[26] Michail Vlachos, George Kollios, and Dimitrios Gunopulos. 2002. Discovering Similar Multidimensional Trajectories. In *Proceedings of the 18th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 673–684. https://doi.org/10.1109/ICDE.2002.994784

[27] Xiaoyue Wang, Abdullah Mueen, Hui Ding, Goce Trajcevski, Peter Scheuermann, and Eamonn J. Keogh. 2013. Experimental Comparison of Representation Methods and Distance Measures for Time Series Data. *Data Mining and Knowledge Discovery* 26, 2 (2013), 275–309. https://doi.org/10.1007/s10618-012-0250-5

[28] Geoffrey I. Webb and François Petitjean. 2021. Tight Lower Bounds for Dynamic Time Warping. *Pattern Recognition* 115 (2021), 107895. https://doi.org/10.1016/j.patcog.2021.107895

[29] Di Yao, Gao Cong, Chao Zhang, Xuying Meng, Rongchang Duan, and Jingping Bi. 2022. A Linear Time Approach to Computing Time Series Similarity Based on Deep Metric Learning. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2022), 4554–4571. https://doi.org/10.1109/TKDE.2020.3047070

[30] Chin-Chia Michael Yeh, Yan Zhu, Liudmila Ulanova, Nurjahan Begum, Yifei Ding, Hoang Anh Dau, Diego Furtado Silva, Abdullah Mueen, and Eamonn Keogh. 2016. Matrix Profile I: All Pairs Similarity Joins for Time Series: A Unifying View That Includes Motifs, Discords and Shapelets. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, Piscataway, NJ, USA, 1317–1322. https://doi.org/10.1109/ICDM.2016.0179

[31] Byoung-Kee Yi, H. V. Jagadish, and Christos Faloutsos. 1998. Efficient Retrieval of Similar Time Sequences under Time Warping. In *Proceedings of the 14th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 201–208. https://doi.org/10.1109/ICDE.1998.655778

**Algorithm 5:** FSMELD_raw$(S, Q, \xi)$

**Input:** **Time series** $S \in \mathbb{R}^n$**, query** $Q \in \mathbb{R}^m$ **and threshold** $\xi > 0$**.**

**Output:** $\{S_{(i)} \mid ELD(S_{(i)}, Q) \leq \xi\}$**.**

$\mathbb{A} \leftarrow \emptyset$ ;

$\ell \leftarrow 2^{\lceil \log_2(8m) \rceil}$ ;

$n_{seg} \leftarrow \lceil \frac{n}{\ell - m + 1} \rceil$; $N_w \leftarrow \ell - m + 1$ ;

**foreach** $i \in \{0, 1, \ldots, n_{seg} - 1\}$ **do**

  $T^{(i)} \leftarrow (S[i \times N_w + 1], S[i \times N_w + 2], \ldots, S[i \times N_w + \ell])$ ;

  $\mathbb{U}^{(i)}, \mathbb{L}^{(i)} \leftarrow$ the upper and lower envelopes for $T^{(i)}$ ;

  $t_U \leftarrow$ OptUpperThreshold$(\mathbb{U}^{(i)}, Q)$ ; // Alg. 2

  $t_L \leftarrow$ OptLowerThreshold$(\mathbb{L}^{(i)}, Q)$ ;

  $A \leftarrow (g_U(1, t_U), g_U(2, t_U), \ldots, g_U(\ell, t_U))$ ;

  $B \leftarrow (h_U(1, t_U), h_U(2, t_U), \ldots, h_U(m, t_U))$ ;

  $\{lb_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A, B)$ ; // Alg. 1

  $A' \leftarrow (g_L(1, t_L), g_L(2, t_L), \ldots, g_L(\ell, t_L))$ ;

  $B' \leftarrow (h_L(1, t_L), h_L(2, t_L), \ldots, h_L(m, t_L))$ ;

  $\{lb'_j\}_{j=1}^{N_w} \leftarrow \mathrm{SIP}(A', B')$ ;

  **foreach** $j \in \{1, 2, ..., N_w\}$ **do**

    **if** $\xi \geq \mathrm{trans}(lb_j + lb'_j) - \delta_{\mathrm{FFT}}$ **then**

      **if** $\xi \geq BGLB(S_{(j+i\times N_w)}, Q)$ **then**

        **if** $\xi \geq ELD(S_{(j+i\times N_w)}, Q)$ **then**

          $\mathbb{A} \leftarrow \mathbb{A} \cup \{j + i \times N_w\}$ ;

**return** $\mathbb{A}$;